

Contents

General Introduction	1
1 Project Context and Background	4
1.1 Introduction	4
1.2 Host Organization: Adria Business & Technology	4
1.2.1 Company Overview and Positioning	4
1.2.2 Vision, Mission, and Core Values	5
1.2.3 Areas of Expertise	6
1.2.4 Information Sheet	6
1.2.5 Products and Services Portfolio	7
1.2.6 Organizational Structure	7
1.2.7 Strategic Partnerships and Clientele	7
1.3 General Context of the Project	8
1.3.1 Supply Chain Finance: Domain Overview	8
1.3.2 Trade Finance as a Complementary Domain	9
1.3.3 Problem Statement: Why Digitizing SCF and Trade Finance	9
1.3.4 Proposed Solution and Platform Ambition	10
1.3.5 Multi-Bank and Configuration Strategy	10
1.4 Project Methodology and Management	11
1.4.1 Agile Framework: Scrum at Adria	11
1.4.2 Sprint Structure and Jira-Based Ticket Management	11
1.4.3 Developer Role and Contribution Scope	12

1.4.4	Tools and Work Environment	12
1.4.5	Timeline and Internship Planning	13
1.5	Conclusion	14
2	System Architecture and Technical Stack	15
2.1	Introduction	15
2.2	Overall Architecture: Microservices on Adria Standard	15
2.2.1	The Adria Standard	15
2.2.2	Microservices Architecture Overview	16
2.2.3	Monorepo Organization: Back-end and Front-end Structure	17
2.2.4	Service Communication and Inter-service Protocols	17
2.3	Backend Services Deep Dive	18
2.3.1	<code>adria-scf-service</code> : Core Business Logic	18
2.3.2	<code>adria-scf-entities</code> : Shared JPA Entity Library	18
2.3.3	<code>adria-gateway2-service</code> : API Gateway	18
2.3.4	Foundation Data Library	19
2.3.5	Properties, Registry, Configuration, and Audit Services	19
2.4	Frontend Architecture	20
2.4.1	Technology Stack	20
2.4.2	Routing Strategy and Role-Based Views	20
2.4.3	Services Layer	20
2.5	Security Architecture	21
2.5.1	Identity and Access Management with Keycloak	21
2.5.2	JWT Validation in the Backend	21
2.5.3	Maker/Checker Governance	22
2.6	Data Storage Architecture	22
2.6.1	Relational Persistence with PostgreSQL	22
2.6.2	Object Storage and Document Conversion	22
2.7	Infrastructure and Deployment	23

2.7.1	Docker Compose Stack	23
2.7.2	Environment-Specific Configuration	24
2.8	Conclusion	25
3	SCF Domain: Design and Implementation	26
3.1	Introduction	26
3.2	SCF Data Model: Entities and Relationships	26
3.2.1	BaseEntity: Shared Auditing and Governance Fields	26
3.2.2	Core Business Entities	27
3.2.3	Business Enumerations	28
3.3	Maker/Checker Governance Pattern	29
3.3.1	Business Rationale	29
3.3.2	Technical Implementation	30
3.3.3	OriginalDataRestorer	30
3.4	Counterparty Module	31
3.4.1	Business Purpose	31
3.4.2	Onboarding Workflow	31
3.4.3	Document Management for Counterparties	31
3.5	Product Definition Module	32
3.5.1	Product Versus Program	32
3.5.2	Product Definition Lifecycle	32
3.6	Program Configuration Module	32
3.6.1	What a Program Represents	32
3.6.2	Program Creation Flow	33
3.6.3	Fee Catalogue Structure	33
3.6.4	Program Copy Feature	34
3.7	Invoice Upload and Cashflow Management	34
3.7.1	Cashflow Validation Rules	34
3.8	Transaction Inquiry Module	35

3.9	Conclusion	35
4	Frontend Implementation and User Flows	36
4.1	Introduction	36
4.2	Application Architecture and Navigation	36
4.2.1	Application Entry Point and Route Structure	36
4.2.2	Role-Based Sidebar and View Adaptation	37
4.2.3	<code>Index.tsx</code> and Module Selection	38
4.3	Master Setup Module	38
4.3.1	Tab Structure and Role-Gated Access	38
4.3.2	Product Suite and Program Configuration	38
4.3.3	Counterparty Onboarding and Foundational Data	39
4.3.4	Control Centre	39
4.4	Invoice Management Flows	39
4.4.1	Invoice Upload Flow	39
4.4.2	Manual Invoice Form	40
4.5	Finance and Early Payment Flows	41
4.5.1	Early Payment Request	41
4.5.2	Finance Disbursement Wizard	41
4.6	Document Inquiry and Specification Documents	42
4.7	Notifications Panel	42
4.8	Forms Engineering: Patterns and Practices	42
4.8.1	Hooks and Form State	42
4.8.2	Error Handling and User Feedback	42
4.9	API Services Layer	43
4.9.1	Axios Client and Endpoint Mapping	43
4.9.2	TypeScript Types	43
4.10	Conclusion	44

5	Disbursement Module: PFE Project	45
5.1	Introduction	45
5.2	Context and Motivation	45
5.2.1	Position in the SCF Value Chain	45
5.2.2	Why the Module Was Treated as Parallel Work	46
5.2.3	Relationship with the Existing Platform	47
5.3	Requirements and Functional Specification	48
5.3.1	Functional Requirements	48
5.3.2	Non-functional Requirements	48
5.3.3	Actors and Use Cases	49
5.4	Design	50
5.4.1	Module Architecture	50
5.4.2	Data Model	51
5.4.3	API Design	51
5.4.4	UML Activity Diagram	52
5.4.5	UML Sequence Diagram	53
5.5	Implementation	54
5.5.1	Frontend Implementation	54
5.5.2	Backend Implementation	55
5.5.3	Key Technical Decisions	55
5.6	Testing and Validation	56
5.7	Conclusion	57
6	Evaluation and Reflection	58
6.1	Introduction	58
6.2	Contributions Summary	58
6.2.1	Sprint-by-Sprint Overview	58
6.2.2	Classification of Contributions	60
6.2.3	Quantitative Summary	60

6.3	Critical Evaluation	61
6.3.1	What Went Well	61
6.3.2	Difficulties Faced and How They Were Handled	62
6.3.3	Limitations of the Current Work	62
6.4	Skills Acquired	63
6.4.1	Technical Skills	63
6.4.2	Professional and Soft Skills	64
6.5	Recommendations for Future Development	64
6.5.1	Integration of the Disbursement Module	64
6.5.2	Asynchronous Processing	65
6.5.3	Mobile Access for Counterparties	65
6.5.4	Advanced Reporting and Analytics	65
6.6	Conclusion	66

List of Figures

1.1	Supply Chain Finance ecosystem	8
1.2	Daily development and validation workflow	11
1.3	Internship timeline and parallel Disbursement Module refinement	14
2.1	Global microservices architecture of the SCF platform	16
2.2	Gateway routing and filter chain	19
2.3	Authentication and role resolution flow	21
2.4	Document storage and conversion flow	23
3.1	SCF core data model	27
3.2	Maker/checker governance lifecycle	30
3.3	Program configuration functional flow	33
4.1	Frontend routing and role-based navigation	37
4.2	Invoice upload component flow	40
4.3	Finance disbursement six-step wizard	41
4.4	Frontend services layer	43
5.1	Disbursement Module position inside the SCF value chain	46
5.2	Roadmap from refinement work to native platform integration	47
5.3	UML activity diagram for the target disbursement request flow	53
5.4	UML sequence diagram for target disbursement submission	54
6.1	Two contribution streams during the internship	59
6.2	Main skills acquired during the internship	63

6.3	Recommendations for future platform development	65
-----	---	----

List of Tables

1.1	Adria Business & Technology information sheet	6
1.2	Tools and work environment	13
2.1	Main infrastructure services used by the platform	24
3.1	Main SCF enumerations and their role	29
4.1	Frontend implementation patterns	44
5.1	Target functional requirements for the Disbursement Module	48
5.2	Non-functional expectations	49
5.3	Actors involved in the Disbursement Module	50
5.4	Target data groups for a finance disbursement	51
5.5	Target API contract candidates	52
5.6	Validation scenarios for the Disbursement Module	56
6.1	Internship timeline and contribution focus	59
6.2	Contribution categories	60
6.3	Quantitative indicators to complete before final submission	61
6.4	Main difficulties and responses	62
6.5	Skills acquired and reinforced	64

General Introduction

The financial relationship between large corporate actors and their commercial partners has become one of the most sensitive points in modern banking operations. Suppliers often need liquidity before invoice maturity. Buyers seek longer payment terms without weakening their commercial network. Banks, positioned between these two forces, must transform trade data into controlled financing operations while preserving security, traceability, regulatory discipline, and operational speed. This is the business terrain in which Supply Chain Finance operates.

Supply Chain Finance, commonly referred to as SCF, covers a family of financing mechanisms designed to optimize working capital across a commercial chain. Its purpose is not limited to granting credit. It restructures the timing of cash movement between anchors, counterparties, and financial institutions. In an SCF program, a bank can rely on the commercial strength of an anchor, the validity of invoices or other underlying instruments, and a set of negotiated rules to provide financing to suppliers, buyers, or distributors. Trade Finance completes this picture by addressing broader instruments and documentary operations linked to commercial exchange. Together, these domains form a demanding digital environment: the system must understand business roles, legal documentation, product definitions, fee rules, approval workflows, and transaction lifecycles.

This end-of-studies internship took place at Adria, from February 2026 to July 1st, 2026, within the Supply Chain Finance Platform Team. The project was conducted in an Agile Scrum context, where development tasks were organized through Jira tickets, source code was managed with Bitbucket, and each contribution followed a pull request review process led by the technical lead. The first two weeks were dedicated to remote integration and training, with a dual focus on the business logic of the Adria architecture and the technical stack used by the team, especially Java and Spring Boot. After this onboarding period, the work continued on-site as part of the daily development rhythm of the platform.

The internship was not limited to observation or peripheral tasks. It was situated inside an active software delivery cycle. The team was working on a Supply Chain Finance and Trade Finance platform intended for banking use cases in Morocco and North Africa. The platform digitizes the financing of trade receivables and payables between corpo-

rate anchors and their counterparties, including suppliers, distributors, and SMEs. Its functional perimeter includes reverse factoring, dynamic discounting, receivables finance, payables finance, and distributor finance. These products are configured through programs that bind together an anchor, counterparties, financing limits, fees, disbursement rules, repayment rules, and governance states.

From a technical perspective, the system follows a microservices-oriented architecture built on Adria standards. These standards provide pre-configured Spring Cloud services and reusable infrastructure patterns that support new products developed inside the company. The platform includes a core SCF service, a gateway service, a foundation data library, a properties service, a registry service, a configuration service, and supporting services for audit, storage, authentication, and document conversion. The backend stack relies mainly on Spring Boot, Spring Security, JPA/Hibernate, PostgreSQL, Eureka, Spring Cloud Config, MinIO, Gotenberg, RabbitMQ, and Keycloak. The frontend is implemented as a React application using Vite, TypeScript, React Router, React Query, Axios, and a component system based on shadcn/ui and Radix UI.

Security and governance represent a major axis of the platform. Authentication is based on Keycloak and JWT tokens, while role extraction determines whether the user operates from a bank, anchor, or counterparty view. Backend access is protected through role-based authorization. Business validation follows the maker/checker principle, a common banking governance pattern where the user who creates or modifies a record cannot approve the same operation. This principle is implemented through approval statuses, pending actions, repository specifications, and rollback mechanisms based on original data snapshots. The same concern for control appears in the gateway layer, which handles request routing, antivirus scanning for uploaded documents, brute-force protection, maintenance mode, request logging, and correlation identifiers.

Within this broader platform, the academic subject of the internship is the Disbursement Module. Its development runs in parallel with the team's sprint work because the main platform had not yet reached the sprint milestone where the module would be integrated natively into the architecture. This situation shaped the project in a very concrete way. The module had to be studied, designed, and progressively prepared while the platform itself was still evolving from Sprint 2 toward Sprint 8. Functional refinement sessions with the Product Owner played a central role in this process: the business rules were discussed, diagrams were prepared, and the proposed flows were evaluated and adjusted according to the expected target architecture.

The central problem addressed by this report is therefore twofold. The first dimension is functional: how can a banking platform organize Supply Chain Finance operations in a way that reflects the realities of anchors, counterparties, invoices, programs, fees,

disbursements, repayments, and approvals? The second dimension is technical: how can such a platform be implemented with a modular architecture that remains secure, maintainable, auditable, and adaptable to several banking contexts? The Disbursement Module stands at the intersection of these two dimensions because it converts validated financing intent into an operational payment flow, while depending on upstream entities such as programs, products, invoices, fee catalogues, and eligibility rules.

This report presents the internship work through six chapters. The first chapter introduces the host organization, the business context of Supply Chain Finance and Trade Finance, the project environment, and the methodology followed during the internship. The second chapter studies the global architecture and technical stack, with attention to the microservices, gateway, security, data storage, and deployment environment. The third chapter examines the SCF domain model and backend implementation, including entities, maker/checker governance, counterparty management, product definitions, program configuration, invoice upload, and transaction inquiry. The fourth chapter focuses on frontend implementation and user flows, from authentication and role-based navigation to invoice, document, early payment, and finance disbursement screens. The fifth chapter is dedicated to the Disbursement Module as the PFE project, covering its motivation, specification, design, implementation choices, and validation. The final chapter evaluates the work performed during the internship, reflects on the acquired skills, and identifies possible future improvements.

The ambition of this document is not merely to list technologies or reproduce development tasks. It aims to analyze the construction of a banking software platform as an engineering problem shaped by business constraints, security expectations, team methodology, and architectural decisions. The work described here was carried out inside a real delivery environment, with all the friction that such an environment implies: evolving requirements, sprint priorities, review cycles, integration constraints, and the need to align academic objectives with production-oriented software practices.

Chapter 1

Project Context and Background

1.1 Introduction

This chapter situates the internship within its professional, business, and methodological environment. A technical project does not emerge from an empty space. It takes shape inside an organization, under a particular delivery model, with a business vocabulary, a set of constraints, and a rhythm imposed by real teams. The Supply Chain Finance platform developed at Adria belongs to this kind of setting: banking software, regulatory sensitivity, iterative delivery, and a domain where each functional term carries operational weight.

The chapter begins with a presentation of Adria Business & Technology as the host organization. It then studies the general context of Supply Chain Finance and Trade Finance, since both domains explain the logic behind the platform and the place of the Disbursement Module inside it. The last part focuses on the working methodology followed during the internship: Scrum ceremonies, Jira-based task tracking, Bitbucket pull requests, technical review, tester validation, and the parallel refinement work conducted with the Product Owner.

1.2 Host Organization: Adria Business & Technology

1.2.1 Company Overview and Positioning

Adria Business & Technology is a Moroccan technology company specialized in software solutions for the financial sector. Its activity is oriented toward banks and financial in-

stitutions that need to modernize their digital channels, automate internal operations, and offer secure services to corporate and retail users. The company operates in a market where reliability is not a decorative quality. A banking platform handles identity, money movement, documents, authorizations, and traces that may later be inspected by operational teams or auditors.

The internship was carried out within Adria's Supply Chain Finance Platform Team. This team works on a platform dedicated to Supply Chain Finance and Trade Finance use cases for Moroccan and North African banks. The platform aims to digitize the financing of commercial relationships between anchors, counterparties, and banks. In practical terms, this means transforming invoices, programs, product definitions, fees, eligibility rules, and approval workflows into executable software processes.

Adria's position is shaped by a double competence. The first is business-oriented: understanding banking workflows, client constraints, financial products, and governance rules. The second is technical: building distributed systems with modern frameworks, reusable company standards, secure integration layers, and maintainable user interfaces. The internship sat exactly between these two worlds. Each technical task had to make sense inside a business flow, and each business rule had to be expressed through code, data structures, APIs, or interface behavior.

1.2.2 Vision, Mission, and Core Values

The information available in the internship context presents Adria as a partner in banking digital transformation. Its mission can be read through the nature of its products: helping financial institutions deliver digital services that are secure, scalable, configurable, and aligned with their operational models. This orientation is visible in the platform architecture, where generic technical standards coexist with banking-specific mechanisms such as maker/checker approval, role-based access, audit, document storage, and gateway-level protection.

The company's working culture, as observed during the internship, also reflects a strong concern for review and validation. Development does not stop at committing code. A ticket is implemented, pushed to Bitbucket, submitted through a pull request, reviewed by the technical lead, adjusted when comments are raised, merged only after acceptance, then sent to testing. The chain is simple when written on paper. Inside a sprint, it demands discipline.

1.2.3 Areas of Expertise

Adria's expertise can be organized around three complementary axes. The first axis is banking domain knowledge, especially digital banking, Trade Finance, payment-oriented services, and corporate banking processes. The second is software engineering methodology, expressed through Agile delivery, collaborative refinement, code review, and progressive validation. The third is technical engineering, with backend, frontend, integration, security, document management, and deployment concerns.

The Supply Chain Finance platform illustrates this combination well. The project requires knowledge of financial products such as reverse factoring and receivables finance, but it also requires an architecture able to route requests through a gateway, validate JWT tokens, store business entities in PostgreSQL, keep documents in MinIO, convert documents through Gotenberg, and expose typed frontend services to a React application. The domain and the stack are inseparable here.

1.2.4 Information Sheet

Table 1.1 summarizes the company information made available through the project guide and internship context.

Table 1.1: Adria Business & Technology information sheet

Item	Information
Company name	Adria Business & Technology
Chief Executive Officer and founder	Mr. BEKKAR Rachid
Legal structure	Limited Liability Company with Sole Proprietor
Business sector	IT consulting and financial software services
Commercial register ID	276409
Company size	51–200 employees
Date of establishment	2013
Address	Shore 26, 2nd Floor, Casa Nearshore Park, Casablanca, Morocco
Contact	contact@adria-bt.com
Website	www.adria-bt.com

1.2.5 Products and Services Portfolio

The example report supplied in the input material identifies several product families associated with Adria's activity: mobile and web banking, mobile wallet, cash management, electronic signature, and Trade Finance solutions. These products share a common direction. They move banking operations from fragmented manual channels toward digital workflows where the client, the bank officer, the back office, and the administration layer can act through controlled interfaces.

The Supply Chain Finance platform belongs to this broader portfolio of financial software. It extends the same logic to corporate financing operations. Rather than treating invoices, counterparties, documents, fees, and approvals as scattered files or manual exchanges, the platform brings them into a structured system where operations can be created, reviewed, approved, queried, and traced.

1.2.6 Organizational Structure

The company structure described in the guide material places general management above several functional divisions, including delivery, integration and support, research and development, commercial activity, and human resources administration. For the internship, the most relevant part of this structure was the technical delivery environment. The intern was not detached from the project team. Work was performed within the Supply Chain Finance Platform Team, under the supervision of a technical lead and in coordination with the Product Owner during refinement activities.

This organizational setting matters because it explains how decisions were made. Functional questions were clarified with the Product Owner. Implementation choices were reviewed by the technical lead. Validation was transferred to testing after merge. The internship therefore exposed the student to the full chain of a real software delivery organization, not only to isolated development.

1.2.7 Strategic Partnerships and Clientele

The available guide material presents Adria as a company collaborating with banks and financial institutions across several countries in Africa, Europe, and Asia. In the context of this report, the point is not to reproduce a marketing catalogue. What matters technically is that such a client base imposes configurability. Banking platforms cannot be rigid copies of a single deployment. They often need profiles, branding, authentication settings, environment variables, data isolation rules, and integration options adapted to each institution.

This expectation appears in the project context through Adria standards and environment-specific configurations. New products rely on pre-configured Spring Cloud services and reusable infrastructure patterns. The platform is therefore designed not only for one development environment, but for a banking product line where different institutional contexts may require different configuration layers.

1.3 General Context of the Project

1.3.1 Supply Chain Finance: Domain Overview

Supply Chain Finance is a set of financing techniques used to improve working capital across commercial relationships. It addresses a frequent tension: suppliers may need early liquidity, while buyers may prefer to preserve longer payment terms. The bank intervenes by transforming approved commercial obligations, often invoices or equivalent instruments, into financing opportunities under a controlled program.

The SCF ecosystem is built around three main actors. The anchor is usually a large buyer or seller whose commercial strength structures the program. The counterparty may be a supplier, buyer, distributor, SME, or corporate partner linked to the anchor. The bank configures the program, validates conditions, finances eligible operations, and supervises the lifecycle. Figure 1.1 presents this ecosystem in a simplified form.

Supply Chain Finance Ecosystem

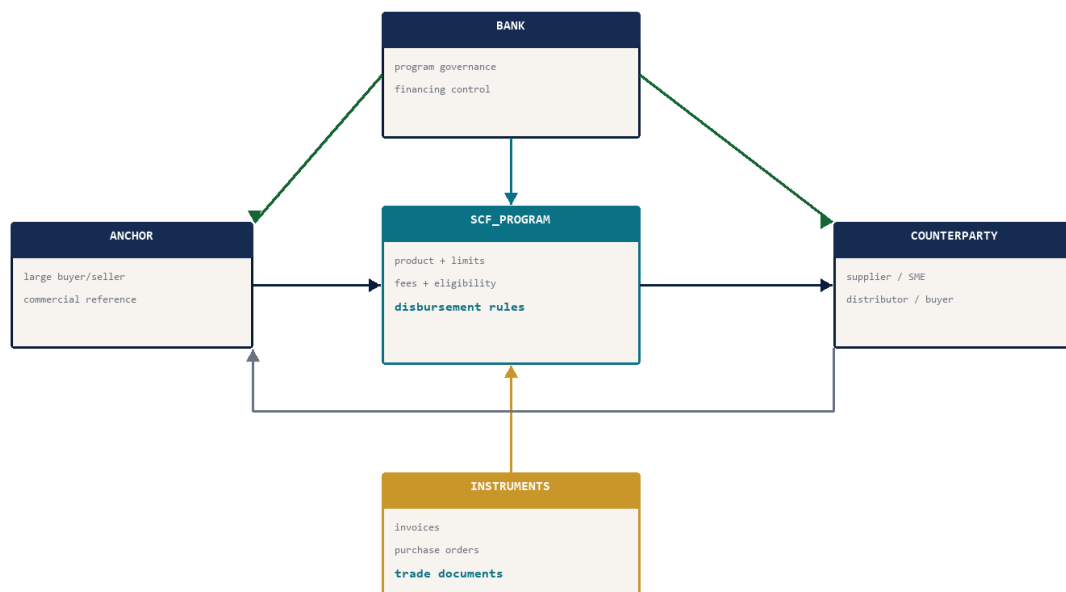


Figure 1.1: Supply Chain Finance ecosystem

The financing logic depends on the product type. In reverse factoring, the supplier can receive early payment based on the buyer’s approved invoice. In dynamic discounting, early payment may be linked to discount rules that vary according to timing. Receivables finance and payables finance address different sides of the commercial obligation. Distributor finance extends the logic toward downstream distribution networks. The platform described in the internship context supports these product families through reusable product definitions and concrete program configurations.

1.3.2 Trade Finance as a Complementary Domain

Trade Finance is broader than SCF. It covers financial instruments and documentary mechanisms used to secure commercial exchanges, including purchase orders, bills of exchange, promissory notes, and letters of credit. The SCF platform context explicitly references Trade Finance because the two domains share several operational concerns: document handling, counterparty identification, validation of underlying instruments, transaction tracking, and governance.

This overlap explains why the project cannot be reduced to a simple invoice screen. It needs a business model capable of representing products, programs, anchors, counterparties, documents, cashflows, fees, statuses, and approval operations. Even when the user interface appears straightforward, the underlying model remains dense.

1.3.3 Problem Statement: Why Digitizing SCF and Trade Finance

The digitization of SCF and Trade Finance responds to several operational problems. Manual handling of invoices and financing requests makes the process slower and more exposed to errors. Scattered documents complicate traceability. Approval decisions become harder to audit when they are not attached to structured records. Product rules may be inconsistently applied if they live outside the system. For banks, these weaknesses are not minor inconveniences; they can affect risk control, client service, and operational visibility.

The platform addresses these issues by moving business operations into a structured digital workflow. Counterparties are onboarded with bank accounts and documents. Product definitions create reusable templates. Programs bind anchors, counterparties, finance limits, fee catalogues, cashflows, disbursement methods, and repayment methods. Maker/checker governance ensures that sensitive records are not approved by their own creator. Gateway-level filters add protection around uploaded files, login attempts,

maintenance mode, and request traceability.

1.3.4 Proposed Solution and Platform Ambition

The proposed solution is a Supply Chain Finance and Trade Finance platform built on a microservices architecture. Its ambition is to provide banks with a configurable environment for managing corporate financing programs and their related operations. The back-end services handle core SCF business logic, reference data, runtime properties, gateway routing, discovery, configuration, audit, storage, and document conversion. The frontend provides role-based access to dashboards, master setup, counterparty onboarding, invoice upload, transaction inquiry, document inquiry, early payment, and finance disbursement flows.

The project context makes an important distinction between product and program. A product definition is a reusable template such as reverse factoring or receivables finance. A program is a concrete instance linked to an anchor, counterparties, currency, dates, limits, fees, disbursement rules, and repayment rules. This distinction is one of the intellectual hinges of the platform: it separates reusable banking product design from the operational configuration used for a specific business relationship.

1.3.5 Multi-Bank and Configuration Strategy

The initial context file contained a broad statement about multi-tenant ambition and audit routing. A correction then clarified the scope that should be retained for this report: Adria teams follow internal standards when starting a new product. These standards include pre-configured Spring Cloud services and multiple environment configurations. Some advanced components, such as the audit service, are handled by another team and remain outside the intern's direct scope, even if they influence the general platform environment.

This distinction is important. The report can analyze the presence of standards without pretending that every standard service was designed or implemented during the internship. The relevant conclusion is more measured: the SCF platform is developed inside an ecosystem of reusable Adria services, and this ecosystem encourages consistency across products through common configuration, gateway, discovery, and infrastructure patterns.

1.4 Project Methodology and Management

1.4.1 Agile Framework: Scrum at Adria

The internship followed an Agile Scrum methodology. The student joined the platform team during Sprint 2 and the project context indicates that the team was entering Sprint 8 at the time the notes were prepared. This means the internship unfolded across an active delivery sequence rather than a frozen specification phase. Requirements, implementation tasks, reviews, and functional discussions moved together.

Scrum gave the work a repeated rhythm. Sprint planning and task assignment defined short-term priorities. Refinement sessions clarified upcoming business needs. Development work was organized around Jira tickets. Pull requests provided a review gate before code reached the shared branch. Testing followed after merge. This rhythm is shown in Figure 1.2.

Daily Development and Validation Workflow

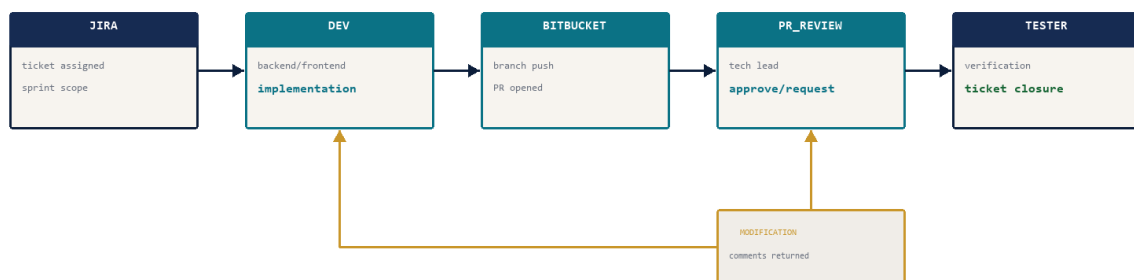


Figure 1.2: Daily development and validation workflow

1.4.2 Sprint Structure and Jira-Based Ticket Management

Each development task was tracked through Jira. The ticket served as a unit of coordination: it described the expected work, provided visibility inside the sprint, and made progress observable by the team. Once the implementation was finished, the code was pushed to Bitbucket and submitted through a pull request. The technical lead reviewed the changes and either requested modifications or merged the branch.

After merge, the ticket moved to the tester for verification. This sequence exposed the intern to a professional feedback loop. Code was not judged only by whether it worked locally. It had to be readable enough for review, compatible with team conventions, and

acceptable for later validation. That pressure is useful. It teaches engineering habits faster than isolated exercises.

1.4.3 Developer Role and Contribution Scope

The internship role combined daily platform tasks with the academic PFE assignment. On the team side, the work involved participation in the existing software development lifecycle, using the same tools and review process as the rest of the developers. On the academic side, the assigned project was the Disbursement Module.

The Disbursement Module was treated as parallel work because the main Supply Chain Finance platform had not yet reached the sprint milestone where this module would be natively injected into the architecture. Development and analysis therefore happened dynamically: during gaps in the sprint cycle, after assigned Jira tickets, and especially through weekly refinement discussions with the Product Owner. This arrangement gave the project a particular character. It was not a finished module dropped into an already stable platform; it was a module being shaped while the platform around it was still growing.

1.4.4 Tools and Work Environment

The working environment combined project management, development, version control, review, and validation tools. Table 1.2 summarizes the tools explicitly identified in the internship context and associated guide material.

Table 1.2: Tools and work environment

Tool or environment	Role during the internship
Jira	Sprint task assignment, ticket tracking, and progress visibility
Bitbucket	Source code hosting, branch publication, and pull request workflow
Pull request review	Technical validation by the team lead before merge
Tester validation	Verification of merged tickets after development review
Java and Spring Boot	Main backend development stack studied during onboarding and used by the platform
React, Vite, and TypeScript	Frontend stack of the SCF application
Keycloak	Identity and access management through roles and JWT tokens
Docker Compose	Local infrastructure provisioning for development services

1.4.5 Timeline and Internship Planning

The internship lasted five months, from February 2026 to July 1st, 2026. The first two weeks were dedicated to remote training. This initial phase covered the business layer of Adria’s architecture and the core technical stack, especially Java and Spring Boot. After that, the work moved fully on-site and became part of the daily sprint rhythm.

Alongside the team assignments, the Disbursement Module followed a parallel design loop. Each Monday, during refinement sessions, the Product Owner explained the business concepts and requirements related to the module. Functional diagrams were then prepared and discussed. The Product Owner evaluated these diagrams, validated them when aligned with the expected architecture, or requested modifications when the flow needed correction. Figure 1.3 presents the main timeline.

Internship Timeline

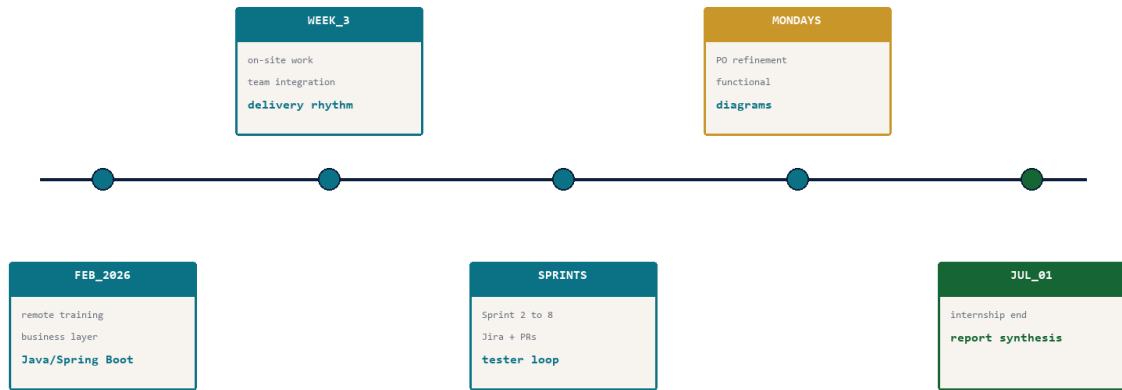


Figure 1.3: Internship timeline and parallel Disbursement Module refinement

This timeline created a productive tension between delivery and analysis. The daily tasks provided exposure to the real platform: entities, endpoints, frontend modules, security rules, review comments, and testing expectations. The Disbursement Module required a more forward-looking effort, since it had to be prepared before its complete integration into the platform roadmap. Both dimensions fed each other. The platform tasks made the academic module more concrete, while the module forced a deeper reading of the SCF value chain.

1.5 Conclusion

This chapter established the environment in which the internship took place. Adria Business & Technology was presented as a financial software company working on banking digital transformation, with the Supply Chain Finance Platform Team providing the direct professional setting. The SCF and Trade Finance context clarified the business reasons behind the platform: financing commercial relationships, structuring programs, governing approvals, managing documents, and giving banks a controlled digital workflow.

The chapter also showed that the internship was organized inside a real Agile delivery process. Jira tickets, Bitbucket branches, pull requests, technical lead review, and tester validation formed the daily development chain. The Disbursement Module, assigned as the PFE project, was developed in parallel with this chain because the platform had not yet reached its native integration sprint for that module. The next chapter moves from context to architecture. It studies the technical structure that supports the platform: microservices, Adria standards, gateway routing, security, storage, frontend architecture, and deployment infrastructure.

Chapter 2

System Architecture and Technical Stack

2.1 Introduction

The previous chapter placed the internship inside its professional and business environment. This chapter moves one level deeper. It studies the technical structure that supports the Supply Chain Finance platform: the services, communication paths, security mechanisms, storage choices, and frontend architecture that allow the business modules to operate as a coherent banking system.

The platform is not a monolithic application hidden behind a single screen. It is a distributed system assembled from several Spring services, a React frontend, a gateway, infrastructure components, identity management, object storage, and development standards already used inside Adria. This architecture gives the project its engineering character. Each user action crosses several boundaries before it becomes a persisted business operation: the browser, the authentication layer, the gateway, the target backend service, and often a database, an object store, or a document conversion service.

2.2 Overall Architecture: Microservices on Adria Standard

2.2.1 The Adria Standard

When a new product starts at Adria, the team does not begin from an empty technical ground. The project follows what the team calls the Adria standard: a set of pre-

configured services, Spring Cloud patterns, environment conventions, and infrastructure habits that allow new banking products to inherit a common operational base. During the internship, this point mattered because the Supply Chain Finance platform was built inside an already structured technical ecosystem.

The correction provided in the project context narrows the interpretation of this standard. Some components, especially the audit service, are part of a wider company initiative and remain outside the direct scope of the intern. They still influence the environment, but they should not be presented as work personally designed during the internship. The relevant observation is more precise: the SCF platform relies on reusable Adria services and conventions, which help teams avoid rebuilding discovery, configuration, gateway behavior, and environment profiles for every product.

2.2.2 Microservices Architecture Overview

The main services identified in the context are the core SCF service, the gateway, the foundation data library, the properties service, the registry service, the configuration service, the audit API, and the React frontend. Around them sit technical infrastructure services such as PostgreSQL, MinIO, Keycloak, RabbitMQ, ClamAV, and Gotenberg. Figure 2.1 gives a compact view of this structure.

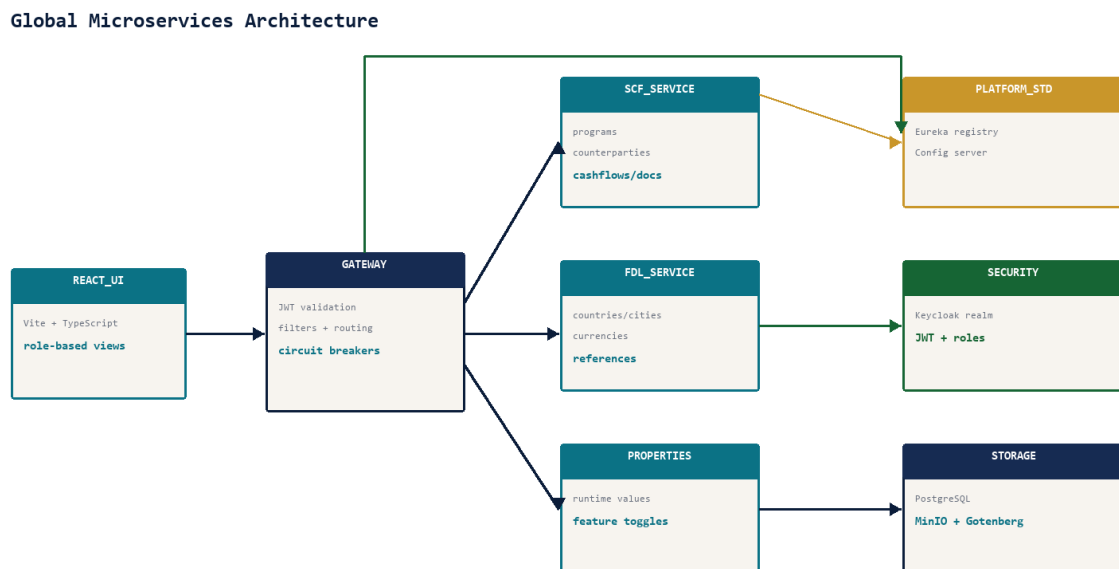


Figure 2.1: Global microservices architecture of the SCF platform

The diagram shows a deliberate separation of concerns. The frontend is responsible for user interaction and role-adapted navigation. The gateway protects and routes external

requests. The SCF service contains the main business logic related to counterparty onboarding, products, programs, invoices, cashflows, and documents. The foundation data library provides shared reference data. The properties service manages runtime configuration values. Infrastructure components complete the picture: service discovery, centralized configuration, identity management, relational persistence, object storage, and document conversion.

2.2.3 Monorepo Organization: Back-end and Front-end Structure

The available context describes a project organized around several backend services and a frontend service named `trade-ui-service`. On the backend side, service boundaries follow functional and technical responsibilities. For example, `adria-scf-service` owns the SCF business modules, while `foundation-data-library-service` handles reference data. Shared entity libraries such as `adria-scf-entities` and `foundation-data-library-entities` provide reusable JPA models without acting as runtime services.

On the frontend side, the React application is organized around pages, feature modules, services, hooks, and typed models. Components such as `SCFMasterSetup`, `SCFCounterPartyOnboarding`, `ProgramFormDialog`, `InvoiceUploadForm`, and `FinanceDisbursementModal` express business features at the interface level. Service files then map user actions to backend endpoints through a shared Axios client.

2.2.4 Service Communication and Inter-service Protocols

External requests enter through the gateway. The configured route prefixes are `/scf/**`, `/fdl/**`, and `/properties/**`. The gateway then forwards requests to downstream services using load-balanced service identifiers such as `lb://adria-scf-service`. Service discovery is provided by Eureka, while centralized configuration is handled through the Config Server.

This routing pattern keeps the frontend from knowing the internal location of each service. The browser calls stable gateway paths; the gateway handles resolution, security filters, fallback behavior, and routing. In a banking platform, that boundary is useful because it concentrates cross-cutting controls at the edge rather than scattering them across every screen.

2.3 Backend Services Deep Dive

2.3.1 `adria-scf-service`: Core Business Logic

The `adria-scf-service` is the center of the domain. It exposes the business operations linked to counterparty onboarding, product definitions, program configuration, invoice or cashflow management, and document metadata. It is implemented with Spring Boot in a blocking servlet model, using JPA/Hibernate for persistence and PostgreSQL as its relational database.

Its internal logic is organized around banking entities and workflows. A counterparty may be created, updated, submitted for approval, approved, rejected, exported, or enriched with documents. A program can be configured with anchors, counterparties, finance limits, fees, disbursement methods, and repayment rules. Cashflows are attached to programs and validated through rules such as duplicate invoice detection, date range consistency, and positive amount checks.

2.3.2 `adria-scf-entities`: Shared JPA Entity Library

The `adria-scf-entities` module contains the shared JPA entity model consumed by the SCF service. It does not act as a running service. Its purpose is structural: it centralizes entity definitions, enumerations, relationships, and common persistence fields. This separation helps keep the domain model reusable while leaving runtime behavior to the service layer.

The entities inherit common fields through `BaseEntity`: technical identity, audit metadata, optimistic locking, approval status, pending action, maker and checker information, and checker comments. This common base is not just a convenience. It encodes the governance model directly into the persistence layer.

2.3.3 `adria-gateway2-service`: API Gateway

The gateway is the first backend component reached by external calls. Its responsibilities include routing, JWT validation, antivirus scanning, brute-force protection, maintenance mode, request logging, correlation identifiers, and circuit-breaker fallback behavior. Figure 2.2 presents the main request path.

Gateway Routing and Filter Chain

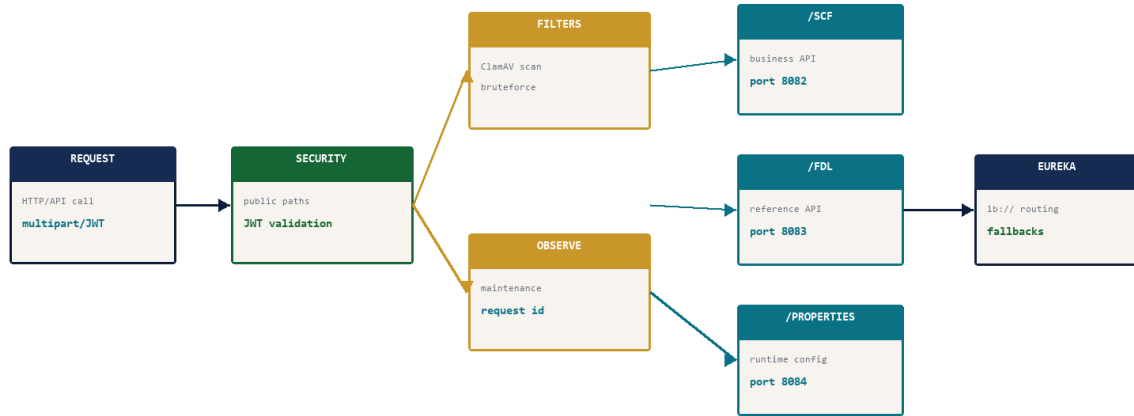


Figure 2.2: Gateway routing and filter chain

The route configuration maps `/scf/**` to `adria-scf-service`, `/fdl/**` to the foundation data library, and `/properties/**` to the properties service. The `lb://` prefix indicates that service resolution is delegated to Eureka. When a downstream service is unavailable, fallback behavior returns a structured error rather than exposing a raw failure to the client.

2.3.4 Foundation Data Library

The foundation data library provides reference and master data used by the platform. Its scope includes countries, cities, states, currencies, bank directories, and transaction reference generation. These records may look secondary compared with financing workflows, yet they are part of the platform’s operational grammar. A program needs currencies. A counterparty needs structured location and bank information. A transaction needs a generated reference that follows a configured format.

The FDL service also follows maker/checker governance for several reference data operations. This consistency matters: banking data governance should not apply only to large business objects while ignoring the reference values that those objects depend on.

2.3.5 Properties, Registry, Configuration, and Audit Services

The properties service manages runtime key-value parameters grouped by module. Examples from the context include fee-related settings, validation parameters, gateway maintenance flags, and audit configuration. The registry service is based on Netflix Eureka and

allows services to register themselves for discovery. The configuration service centralizes external configuration, loaded by microservices at startup.

The audit service is identified as a cross-cutting module dealing with traceability, event capture, persistence strategies, retention, export, and integrity verification. The project correction states that this service is handled by another team and is outside the intern's direct scope. It is still useful to mention because it reflects the broader architecture in which SCF evolves, but the report should avoid attributing its design to the internship work.

2.4 Frontend Architecture

2.4.1 Technology Stack

The frontend is implemented as a React 18 single-page application built with Vite and TypeScript. It uses React Router v6 for navigation, React Query for server-state management, Axios for HTTP communication, and a component system based on shadcn/ui and Radix UI. The use of TypeScript is especially valuable in a platform with many domain models: invoices, programs, counterparties, disbursements, fees, accounting entries, and transaction statuses all benefit from explicit types.

2.4.2 Routing Strategy and Role-Based Views

The application routes include `/auth`, the main protected route `/`, and `/specification-document`. The protected pages are wrapped in an authentication guard. If the user is not authenticated, the guard redirects to the authentication page. If the user is authenticated, the application decodes the token, extracts roles, and determines the business view.

The role resolution logic maps Keycloak roles to one of three views: `BANK`, `ANCHOR`, or `COUNTERPARTY`. This decision shapes the visible sidebar items, dashboard cards, and available actions. A bank user can access administration and master setup features. An anchor user focuses on invoice upload, transaction inquiry, and early payment. A counterparty user sees only the flows relevant to its own financing position.

2.4.3 Services Layer

The frontend services layer wraps the shared Axios client. Endpoint prefixes mirror the gateway routes: `/scf/` for SCF business operations and `/fdl/` for reference data. For ex-

ample, `scfCounterPartyService` maps to `/scf/counterparties`, `scfProgramConfigService` maps to `/scf/programs`, and `cityMasterService` maps to `/fdl/cities`.

This layer gives the interface a typed boundary. Components do not need to construct raw URLs everywhere. They call service functions that express intent: create a counterparty, submit a program, upload invoices, request early payment, or fetch transaction inquiry rows.

2.5 Security Architecture

2.5.1 Identity and Access Management with Keycloak

Authentication is based on Keycloak. The frontend uses a PKCE-based OpenID Connect flow. The browser redirects the user to Keycloak, receives an authorization code after authentication, exchanges it for tokens, stores the access and refresh tokens in session storage, and then decodes the JWT to extract user information and roles.

Figure 2.3 summarizes this path from login to backend authorization.

Authentication and Role Resolution Flow

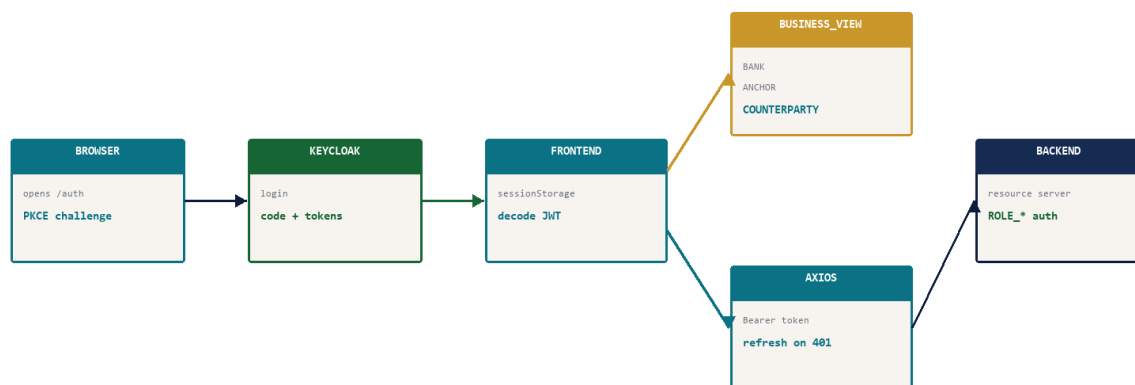


Figure 2.3: Authentication and role resolution flow

2.5.2 JWT Validation in the Backend

Backend services are protected as OAuth2 resource servers. JWT validation relies on the Keycloak issuer and JWKS endpoint. The custom authentication converter extracts roles from `realm_access.roles` and `resource_access.<clientId>.roles`, prefixes them with `ROLE_`, and exposes them as Spring Security authorities.

This mechanism connects identity to authorization. It allows controller methods to enforce rules such as access by bank administrators, makers, or checkers. It also supports the frontend’s role-adapted behavior by using the same token as the source of truth for the user’s business view.

2.5.3 Maker/Checker Governance

The maker/checker principle is one of the strongest banking patterns in the platform. A user who creates or modifies a sensitive record cannot approve the same change. Technically, this is enforced through approval statuses, pending actions, maker and checker metadata, and repository specifications that exclude a maker’s own pending records from the checker queue.

The mechanism is completed by rollback support. Before an update, the original entity state is serialized into the `originalData` field. If the checker rejects the change, the system can restore the previous state. This design makes approval more than a status flag; it becomes a controlled transition with traceable intent and recoverable state.

2.6 Data Storage Architecture

2.6.1 Relational Persistence with PostgreSQL

PostgreSQL is used as the main relational database for business services. The SCF service persists entities such as counterparties, bank accounts, documents, product definitions, program configurations, program anchors, program counterparties, fee catalogues, and cashflows. The foundation data library uses PostgreSQL as well for reference data and transaction reference generation.

Relational persistence is well suited here because many platform concepts have strong identity and relationship requirements. A program is linked to anchors and counterparties. A counterparty owns bank accounts and documents. A cashflow belongs to a program. These relationships must be queryable, validated, and protected by transactional behavior.

2.6.2 Object Storage and Document Conversion

Documents are stored through a port/adaptor pattern. The `ObjectStoragePort` defines operations such as upload, download, delete, presigned URL generation, bucket existence check, and bucket creation. The MinIO adapter implements these operations with the

MinIO Java SDK. The SCF service stores document metadata in PostgreSQL while the binary content lives in MinIO.

Gotenberg handles document conversion, especially office-document-to-PDF conversion through LibreOffice endpoints. Figure 2.4 shows the upload and conversion flow.

Document Storage and Conversion Flow



Figure 2.4: Document storage and conversion flow

This separation is clean. PostgreSQL keeps structured metadata that can be searched and governed; MinIO stores file bytes; Gotenberg performs conversion work that should not be embedded directly in the core business service.

2.7 Infrastructure and Deployment

2.7.1 Docker Compose Stack

The development environment is provisioned through Docker Compose. The context lists PostgreSQL, MinIO, Keycloak, RabbitMQ, ClamAV, Gotenberg, Eureka, Config Server, the gateway, the SCF service, the foundation data library, the properties service, and the audit service. These services communicate over a Docker bridge network.

Table 2.1: Main infrastructure services used by the platform

Service	Role	Port
PostgreSQL	Relational persistence for business and reference data	5432
MinIO	Object storage for documents and uploaded files	9000/9001
Keycloak	Identity and access management	8080
RabbitMQ	Messaging infrastructure used by audit-related flows	5672/15672
ClamAV	Antivirus scanning for uploaded multipart files	3310
Gotenberg	Document conversion to PDF or image formats	3000
Eureka	Service discovery registry	8761
Config Server	Centralized external configuration	8888
Gateway	Request routing, filtering, and security edge	8085
SCF service	Core Supply Chain Finance business service	8082
Foundation data library	Reference data and transaction references	8083
Properties service	Runtime properties and feature values	8084

2.7.2 Environment-Specific Configuration

The platform uses environment-specific configuration layers. Microservices load embedded configuration, then retrieve overrides from the Config Server, and may also receive final values from environment variables. This structure makes the same codebase adaptable to different deployment contexts. It also matches the Adria standard discussed earlier: a new product should inherit enough structure to remain coherent with other products, while still allowing environment-level adaptation.

2.8 Conclusion

This chapter presented the technical architecture of the Supply Chain Finance platform. The system is organized around a React frontend, a Spring Cloud gateway, business and reference-data services, centralized configuration, service discovery, Keycloak-based security, relational storage, object storage, and document conversion. The architecture is not accidental; it reflects the needs of a banking platform where traceability, controlled access, modularity, and operational configuration matter as much as feature delivery.

The next chapter narrows the focus from architecture to domain implementation. It studies the SCF data model, maker/checker governance, counterparty onboarding, product definition, program configuration, invoice upload, and transaction inquiry modules.

Chapter 3

SCF Domain: Design and Implementation

3.1 Introduction

Chapter II described the technical architecture that supports the platform. This chapter enters the business core of the system: the Supply Chain Finance domain model and its backend implementation. The aim is to explain how banking concepts such as counterparties, anchors, products, programs, invoices, fee catalogues, documents, and approval states become structured software objects.

The SCF service is not a collection of isolated controllers. It is organized around a domain where each entity participates in a larger financing chain. A counterparty may be onboarded and governed. A product definition describes a reusable financing template. A program turns that template into a concrete operating arrangement between an anchor, counterparties, limits, fees, cashflows, and disbursement rules. Documents and invoices then enrich that program with operational evidence.

3.2 SCF Data Model: Entities and Relationships

3.2.1 BaseEntity: Shared Auditing and Governance Fields

All major SCF entities inherit from a shared **BaseEntity**. This base class provides the technical identity of each record, but its role goes further than an identifier. It carries fields for creation metadata, modification metadata, optimistic locking through a version field, approval status, pending action, maker user, checker user, and checker comments.

This design pushes governance into the entity model itself. Rather than treating approval as a temporary screen state, the platform stores it as part of the persistent record. That choice fits the banking context: a record is not only data; it also has an author, a lifecycle, a review status, and sometimes a rollback history.

3.2.2 Core Business Entities

The core model revolves around counterparties, anchors, product definitions, program configurations, fee catalogues, cashflows, and documents. Figure 3.1 summarizes the main structural relationships.

SCF Core Data Model

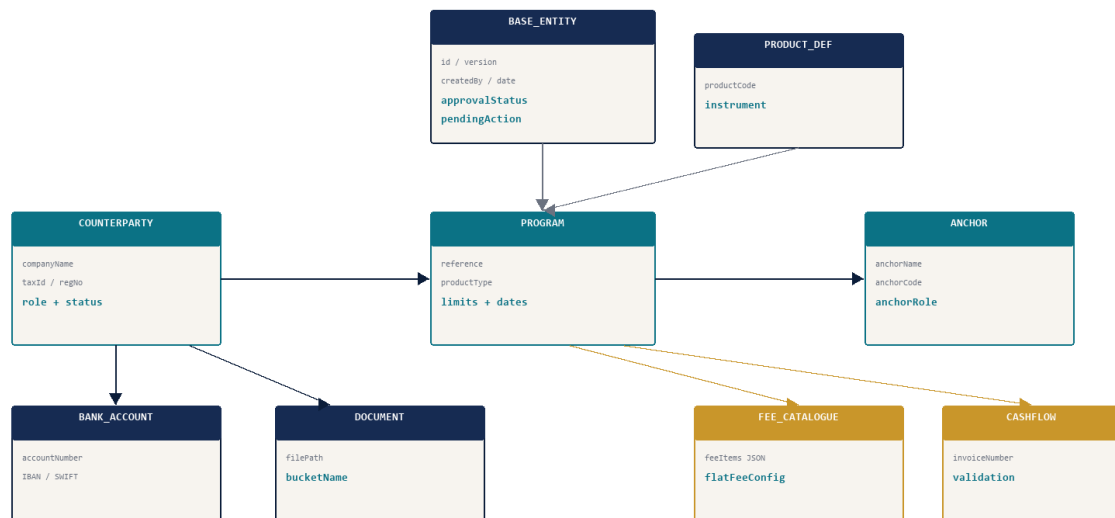


Figure 3.1: SCF core data model

ScfCounterParty represents a corporate, individual, or SME actor participating in trade finance. It stores company identity, legal identity, tax and registration information, contact information, address data, role, status, and rollback data. It is linked to **ScfBankAccount**, which stores banking details such as account number, IBAN, SWIFT code, currency, and primary-account flag. It is also linked to **ScfCounterPartyDocument**, which stores uploaded document metadata while the actual file bytes are kept in MinIO.

ScfAnchor represents the large corporate actor that structures a financing program. In many SCF products, the anchor's commercial strength is what gives the program its risk profile. The entity is linked to **ScfProgramAnchor**, a junction object that binds an anchor to a program with a program-specific role.

ScfProductDefinition is a reusable financing product template. It contains a product name, product code, product type, underlying instrument, description, and status. Its

importance lies in the separation it creates: a product is generic, while a program is concrete.

`ScfProgramConfiguration` is the operational instance. It carries the program name, reference, dates, currency, finance limits, finance percentage, disbursement method, repayment method, grace period, penalty rate, discount rate, and lifecycle status. It is linked to anchors, counterparties, fee catalogues, and cashflows. This entity becomes the center of program execution.

`ScfProgramFeeCatalogue` contains fee configuration. Its design uses JSON-serialized structures: `feeItems`, converted through `FeeItemsJsonConverter`, and `flatFeeConfig`, converted through `FlatFeeConfigJsonConverter`. That design gives the program flexible fee structures without multiplying relational tables for every fee variation.

`ScfProgramCashflow` represents an invoice or cashflow entry inside a program. It stores invoice number, supplier, buyer, amount, currency, invoice date, due date, payment terms, validation status, validation errors, and validation warnings.

3.2.3 Business Enumerations

The entity model is supported by enumerations that constrain business values. Table 3.1 gives a compact reading of the most important ones.

Table 3.1: Main SCF enumerations and their role

Enumeration	Business role
<code>ProductType</code>	Defines supported product families such as reverse factoring, dynamic discounting, receivables finance, payables finance, and distributor finance.
<code>UnderlyingInstrument</code>	Identifies the financial instrument behind the product, such as invoice, purchase order, bill of exchange, promissory note, or letter of credit.
<code>CounterPartyStatus</code>	Tracks the operational status of a counterparty from draft and approval stages to active, suspended, closed, or rejected states.
<code>ApprovalStatus</code>	Represents the governance state used by the maker/checker workflow.
<code>PendingAction</code>	Records the intended operation waiting for checker review: create, update, delete, or none.
<code>ProgramStatus</code>	Controls the lifecycle of a program configuration, including draft, pending approval, approved, active, suspended, closed, and rejected states.

Enumerations are more than code cleanliness. They protect the platform from ambiguous state values and make lifecycle transitions easier to reason about. In a financial system, that is not cosmetic. A free-text status is an invitation to inconsistency.

3.3 Maker/Checker Governance Pattern

3.3.1 Business Rationale

The maker/checker mechanism applies the four-eyes principle to sensitive banking operations. A maker creates or modifies a record. A checker reviews it. The same user cannot play both roles on the same pending operation. This protects the platform against unilateral changes and creates an auditable division between input and approval.

The mechanism appears across several modules: counterparties, product definitions, program configurations, and reference data. It is therefore not a local trick implemented for one screen. It is a recurring governance pattern in the platform.

3.3.2 Technical Implementation

Technically, the pattern is supported by shared entity fields and repository specifications. The `GovernanceSpecifications.excludeOwnPendingRecords(currentUser)` rule prevents a maker's own pending records from appearing in the checker's approval queue. Additional specifications filter by approval status and pending action.

Figure 3.2 presents the lifecycle in simplified form.

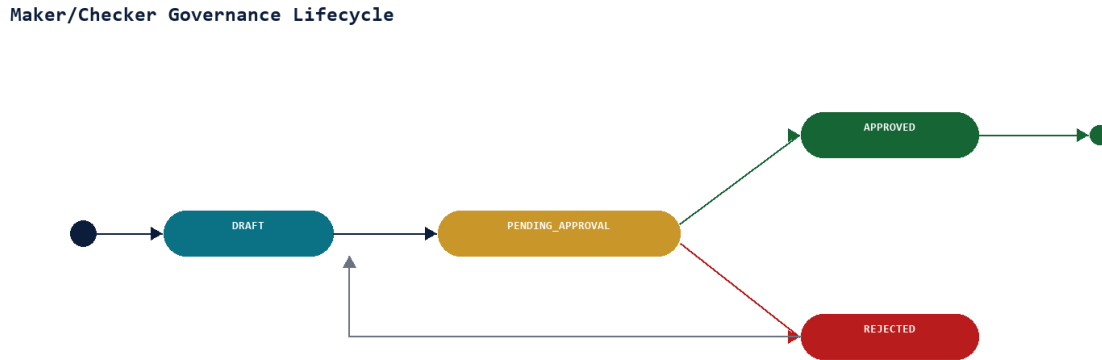


Figure 3.2: Maker/checker governance lifecycle

On creation, a record starts in draft form. When the maker submits it, the record becomes pending approval and carries the corresponding pending action. The checker can approve the operation, which validates the change, or reject it with comments. For updates, rejection can trigger a rollback through `OriginalDataRestorer`.

3.3.3 OriginalDataRestorer

`OriginalDataRestorer` serializes the previous entity state into the `originalData` field before modification. If the checker rejects the change, the stored snapshot is deserialized and applied back to the entity. This gives rejection a material effect. The record does not simply receive a rejected label; it can return to its previous approved state.

The mechanism is especially useful for update and delete requests. A pending update should not destroy the last approved business version until a checker accepts it. By preserving the original data, the system keeps both intent and recoverability.

3.4 Counterparty Module

3.4.1 Business Purpose

The counterparty module manages actors that participate in financing programs. A counterparty can be a supplier, buyer, SME, or corporate actor. It must be identified, described, linked to bank accounts, enriched with documents, and governed through approval.

This module is one of the first operational gates in the SCF value chain. A program cannot reliably finance an invoice if the counterparties behind that invoice are poorly identified. The counterparty record therefore becomes a master object for later program configuration and transaction processing.

3.4.2 Onboarding Workflow

The controller package contains `CounterPartyController` and `CounterPartyDocumentController`. The main controller exposes operations for paginated listing, single fetch, creation, update, deletion, submission, approval, rejection, bulk creation, and CSV export. The document controller manages upload, listing, viewing through presigned URLs, direct download, and deletion.

The onboarding workflow follows a disciplined sequence. A bank maker creates a draft counterparty with company data, tax or registration identifiers, contact information, address fields, and bank accounts. The maker submits the record. A bank checker, different from the maker, reviews the pending operation and approves or rejects it.

3.4.3 Document Management for Counterparties

Counterparty documents are uploaded through multipart requests. The gateway can scan the file through ClamAV before the request reaches the SCF service. The service then stores file bytes through the object storage port and saves metadata in `ScfCounterPartyDocument`. Viewing can use a presigned URL, while download can stream the file directly.

The design keeps business metadata separate from binary storage. PostgreSQL stores the document record; MinIO stores the actual object. This separation avoids loading heavy files into relational rows and keeps retrieval logic explicit.

3.5 Product Definition Module

3.5.1 Product Versus Program

The product definition module manages reusable templates. A product definition states what type of financing product is available and what underlying instrument supports it. A program configuration, by contrast, binds a product idea to concrete parties, limits, dates, fees, and disbursement rules.

This distinction avoids duplication. The bank can define a product category once, then configure multiple programs from it. The implementation reflects this separation through `ScfProductDefinitionServiceImpl`, the product mapper, the product repository, and product-specific exceptions such as duplicate code or unsupported ordering.

3.5.2 Product Definition Lifecycle

Product definitions follow a CRUD and maker/checker lifecycle. The platform validates uniqueness of the product code and manages status transitions through governance fields. In business terms, this prevents uncontrolled product activation. In technical terms, it keeps product metadata consistent before programs depend on it.

3.6 Program Configuration Module

3.6.1 What a Program Represents

A program is the executable configuration of a financing relationship. It combines the product family, the anchor, counterparties, finance limits, currency, dates, fee structure, disbursement method, repayment method, and lifecycle status. It is the point where generic SCF theory becomes a concrete operational arrangement.

The OpenAPI file confirms a step-based program configuration style: general information, cashflow or disbursement-repayment settings, fee catalogue, submission, and copy. The implementation context also identifies `ProgramConfigurationController`, `CashflowUpdateController`, `FeeCatalogueUpdateController`, `ProgramConfigurationService`, `CashflowUpdateService`, `FeeCatalogueUpdateService`, and `ProgramSnapshotService`.

3.6.2 Program Creation Flow

Figure 3.3 summarizes the program configuration flow as a sequence of functional steps.

Program Configuration Functional Flow

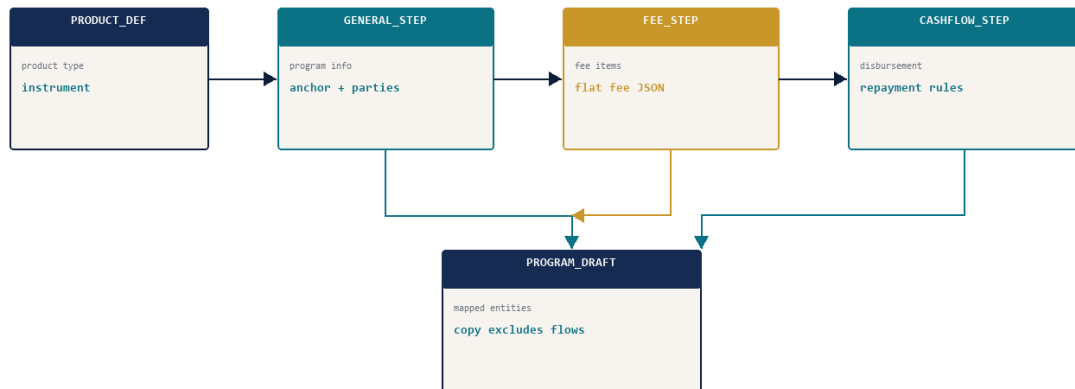


Figure 3.3: Program configuration functional flow

The first step gathers general data: program name, reference, dates, currency, product type, finance limits, anchor assignment, and counterparty assignment. The fee step configures fee items and flat fee settings. The cashflow step configures disbursement and repayment behavior. Once the draft is complete, the program can be submitted into the approval lifecycle.

3.6.3 Fee Catalogue Structure

The fee catalogue is one of the more flexible parts of the model. Each `ProgramFeeItem` can include fee type, calculation basis, rate, flat amount, currency, and frequency. The `FlatFeeConfig` stores flat-fee-level settings. Both are serialized as JSON through dedicated converters.

This approach accepts the variability of banking fees without forcing every possible fee structure into rigid columns. The trade-off is that validation must be handled carefully in the service layer, because JSON flexibility can otherwise hide malformed business data.

3.6.4 Program Copy Feature

`ProgramSnapshotService` supports copying an existing program. It duplicates the configuration and its sub-entities, including anchors, counterparties, and fee catalogues, while excluding cashflows. The copied program starts in draft status.

That exclusion is sensible. Cashflows represent operational entries linked to a particular program life. Copying them into a new draft could create false business history. The feature therefore copies reusable configuration, not transactional evidence.

3.7 Invoice Upload and Cashflow Management

3.7.1 Cashflow Validation Rules

Cashflows represent invoice-level data inside a program. The `CashflowUpdateController` accepts invoice details through `ProgramCashflowRequest`. The service validates duplicate invoice numbers, date consistency, and positive amounts. The result is stored as a `ScfProgramCashflow` with validation status, errors, and warnings.

Algorithm 1: Cashflow validation routine

Data: Program identifier and submitted invoice data

Result: Persisted cashflow with validation status

Load the target program configuration

Check whether the invoice number already exists for the program

Verify that invoice date is not later than due date

Verify that invoice amount is positive

if *blocking validation errors exist* **then**

 | Store the entry with error status and validation details

else

 | Store the entry as valid or warning depending on non-blocking findings

The important point is not only that validation exists. It is that validation results are persisted. This allows the frontend to display row-level feedback, summaries, and rejection reports instead of losing the reason behind a failed invoice.

3.8 Transaction Inquiry Module

The transaction inquiry module provides filtering and consultation of operational records. On the frontend side, `SCFTransactionInquiry` displays filters and a transaction table. Row actions can open invoice or finance disbursement views. On the backend side, this module depends on the consistency of the domain model: transactions are meaningful only if programs, counterparties, invoices, and disbursement records have been structured properly upstream.

The module is therefore a reading surface over the work performed by the other modules. It gives bank officers, anchors, or counterparties a controlled way to inspect the state of financing operations.

3.9 Conclusion

This chapter analyzed the SCF domain model and the backend modules that implement it. The data model is built around counterparties, anchors, product definitions, program configurations, fee catalogues, cashflows, and documents. Governance is embedded through shared entity fields, approval statuses, pending actions, repository specifications, and rollback snapshots. The program module stands at the center of the system because it turns reusable products into concrete financing arrangements.

The next chapter shifts from backend domain implementation to frontend implementation. It studies how these business concepts are exposed through React components, role-based navigation, forms, wizards, service calls, and user-facing flows.

Chapter 4

Frontend Implementation and User Flows

4.1 Introduction

The previous chapter studied the backend domain model and the business modules that support Supply Chain Finance operations. This chapter moves to the frontend layer, where those domain concepts become usable screens, forms, tables, wizards, modals, and navigation flows. A financing platform may have a strong backend model, but users experience it through interaction: selecting a program, uploading invoices, submitting a request, reviewing validation errors, or opening a transaction detail view.

The frontend of the SCF platform is implemented as a React single-page application built with Vite and TypeScript. It relies on React Router for navigation, React Query for server-state handling, Axios for API calls, Keycloak for authentication, and a UI component base built around shadcn/ui and Radix UI. The implementation is organized around feature modules such as master setup, counterparty onboarding, invoice upload, finance disbursement, early payment, transaction inquiry, document inquiry, and foundational data management.

4.2 Application Architecture and Navigation

4.2.1 Application Entry Point and Route Structure

The application starts from `App.tsx`. This file assembles the global providers and route structure. The available routes include `/auth`, the main protected route `/`, and `/specification-document`.

The application is wrapped with `KeycloakAuthProvider`, `QueryClientProvider`, `Toaster`, and `TooltipProvider`. This arrangement gives the interface access to authentication state, server-state caching, user feedback, and tooltip behavior from the top of the component tree.

The main route is protected by `AuthGuard`. If authentication is still loading, the guard displays a loading state. If the user is not authenticated, it redirects to the authentication page. If the user is authenticated, it allows the main interface to render.

4.2.2 Role-Based Sidebar and View Adaptation

The role system is one of the frontend’s central mechanisms. The decoded Keycloak token contains realm roles. The helper `businessViewFromRoles.ts` reads these roles and derives one of three business views: `BANK`, `ANCHOR`, or `COUNTERPARTY`. The `useBusinessView` hook exposes the resulting view and convenience flags such as `isBank`, `isAnchor`, and `isCounterparty`.

Figure 4.1 shows how routing, authentication, and business-view adaptation fit together.

Frontend Routing and Role-Based Navigation

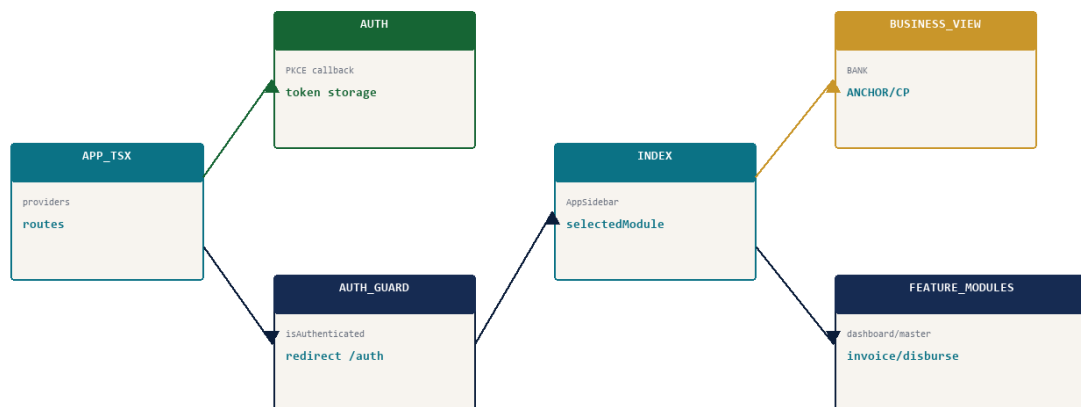


Figure 4.1: Frontend routing and role-based navigation

The `AppSidebar` filters menu items according to the business view. Bank users see the broadest set of modules, including master setup, transaction inquiry, document inquiry, invoice management, early payment, and finance disbursement. Anchor users see a narrower set focused on their own operational activities, such as invoice upload, manual invoice entry, transaction inquiry, and early payment. Counterparty users receive an even more focused view, centered on their own invoices, transactions, and payment requests.

[Insert screenshot of the SCF dashboard with role-based sidebar]

4.2.3 `Index.tsx` and Module Selection

The `Index.tsx` page renders the sidebar and a content area driven by a `selectedModule` state. This state decides which functional component is displayed: dashboard, master setup, transaction inquiry, document inquiry, invoice upload, invoice form, finance disbursement, early payment, or notifications. This approach keeps the primary workspace inside a single routed page while still allowing the user to move between modules.

The design is pragmatic. It avoids turning every module switch into a full route transition, while the application still keeps authentication and global providers outside the page. For an operational banking interface, this feels natural: the user remains inside a controlled workspace and changes the working panel from the sidebar.

4.3 Master Setup Module

4.3.1 Tab Structure and Role-Gated Access

`SCFMasterSetup` acts as a container for several administrative and configuration modules. Its tabs include product suite, program configuration, counterparty onboarding, foundational data, and control centre features. Access to these tabs is role-gated, which prevents users outside the bank view from reaching administrative functions that do not belong to their business position.

[Insert screenshot of the Master Setup tabs]

4.3.2 Product Suite and Program Configuration

The product suite exposes product definition management through `SCFProductSuite`, `SCFProductDefinition`, and product-related components such as `ProductSolutionToggle`. This interface allows the bank side to view, create, or edit product templates according to the supported SCF product families.

Program configuration is handled by `SCFProgramConfiguration` and `ProgramFormDialog`. The dialog is structured as a multi-step configuration process: `GeneralPartyPane`, `FeeCataloguePane`, and `DisbursementRepaymentPane`. The `useProgramForm` hook centralizes the form state and helps keep the wizard logic outside individual panes.

[Insert screenshot of the Program Configuration wizard]

4.3.3 Counterparty Onboarding and Foundational Data

SCFCounterPartyOnboarding provides table-based management of counterparties, inline creation and editing, document upload, maker/checker actions, bulk creation, and CSV export. The component is connected to backend operations through **scfCounterPartyService**.

[Insert screenshot of Counterparty Onboarding table and actions]

Foundational data screens such as **CityMaster**, **CountryMaster**, **CurrencyMaster**, and **StateMaster** provide CRUD interfaces for reference values. These screens may appear less complex than program configuration, but they support the integrity of the wider system. A program or counterparty form depends on stable reference data.

4.3.4 Control Centre

The control centre groups configurable UI and field behavior modules. **FieldDefinition** manages custom field definitions. **FieldActionsTab** configures action rules at field level. **ManagePanelsAndSections** handles layout configuration. **ProductEventMapping** connects product events to platform behavior. These modules show that the frontend is not only a static set of forms; part of its purpose is to expose controlled configuration surfaces to bank administrators.

4.4 Invoice Management Flows

4.4.1 Invoice Upload Flow

Invoice upload is orchestrated by **InvoiceUploadForm**. The component coordinates the Excel uploader, scanned invoice uploader, validation table, summary cards, disbursement eligibility display, rejection report download, progress state, and error display. Figure 4.2 summarizes the component flow.

Invoice Upload Component Flow

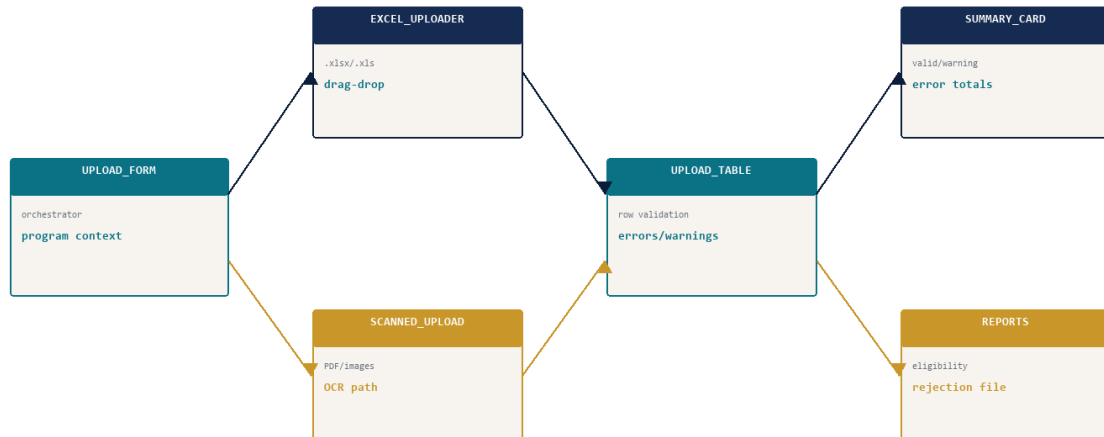


Figure 4.2: Invoice upload component flow

The Excel path uses `ExcelInvoiceUploader` for spreadsheet files. The scanned path uses `ScannedInvoiceUploader` for images or PDF files that may require OCR processing. Once uploaded, rows are displayed in `InvoiceUploadTable`, with validation status and row-level issues. `ValidationSummaryCard` then aggregates the result, while `DisbursementStatusCard` gives an eligibility-oriented view.

[Insert screenshot of Invoice Upload validation table]

4.4.2 Manual Invoice Form

The manual invoice flow is handled by `InvoiceForm`. It is organized into panes: `InvoiceGeneralDetails`, `InvoiceLineItemsPane`, and `InvoiceSummaryPane`. Supporting components such as `InvoiceProgressIndicator`, `InvoicePaneRenderer`, and `InvoiceFormActions` give structure to the form experience.

The `useInvoiceForm` hook centralizes state and calculation logic. This pattern keeps the panes focused on presentation and interaction rather than forcing each one to carry the whole form model. For multi-pane forms, that distinction helps readability.

4.5 Finance and Early Payment Flows

4.5.1 Early Payment Request

The early payment flow is implemented through `EarlyPaymentRequestModal` and `EarlyPaymentRequestService`. It captures the requested payment date, whether the discount is accepted, and the calculated net amount. The operation connects to `earlyPaymentService`, which maps the frontend action to the backend endpoint.

This flow is smaller than the disbursement wizard, but it is highly meaningful in SCF. Early payment is one of the places where a commercial obligation becomes a financing decision, and the interface must make timing, discount, and acceptance clear to the user.

4.5.2 Finance Disbursement Wizard

The finance disbursement flow is implemented as a six-step wizard inside `FinanceDisbursementModal`. The panes are `ProgramProductSelectionPane`, `InvoiceSelectionPane`, `FinanceDetailsPane`, `AccountingEntriesPane`, `RepaymentDetailsPane`, and `ReviewSubmitPane`. Figure 4.3 presents the sequence.

Finance Disbursement Six-Step Wizard



Figure 4.3: Finance disbursement six-step wizard

The first pane selects the program and product context. The invoice pane loads eligible invoices and allows the user to select which invoices will be financed. The finance details pane captures terms and presents fee breakdown information. Accounting entries are displayed as debit and credit lines. The repayment pane configures schedule-related details. The final pane reviews the full request before submission.

[Insert screenshot of Finance Disbursement wizard]

4.6 Document Inquiry and Specification Documents

`DocumentInquiry` gives users a filtered browsing interface for documents. `DocumentUploadPopup` handles file selection and upload behavior, while `DocumentUploadDetails` supports viewing and downloading. The same document concerns described in earlier chapters reappear here in interface form: the user does not see `MinIO` or presigned URL generation directly, but the UI must expose a smooth document lifecycle.

The `SpecificationDocument` page is routed separately and protected through `AuthGuard`. It provides a dedicated surface for specification-related documents, which is useful in a platform where functional rules, API specifications, and operational references may evolve alongside development.

4.7 Notifications Panel

`NotificationsPanel` provides a place for user-facing events and alerts. Although the current context does not describe a complete notification backend, the component is part of the frontend structure. In an SCF platform, notifications naturally belong near approval events, validation outcomes, document status changes, and financing decisions.

4.8 Forms Engineering: Patterns and Practices

4.8.1 Hooks and Form State

Several frontend flows are too large to be kept inside a single component without becoming fragile. The project uses custom hooks such as `useInvoiceForm`, `useProgramForm`, `useBusinessView`, `useAuth`, and `useFieldActions`. These hooks separate reusable behavior from visual components.

4.8.2 Error Handling and User Feedback

The application includes Sonner-based toast feedback through the global `Toaster`. API errors are supported by helper logic such as `scfApiError.ts`. This matters because banking forms are validation-heavy. A failed upload, a rejected field, or an expired token should not leave the user guessing.

4.9 API Services Layer

4.9.1 Axios Client and Endpoint Mapping

The shared Axios client is defined under `integrations/gateway/client.ts`. It reads the access token from session storage and attaches it as a bearer token. On unauthorized responses, it attempts refresh behavior and logs the user out if the session cannot be recovered.

Feature services wrap this client with typed functions. Examples include `scfProgramConfigService`, `scfProductService`, `invoiceUploadService`, `financeDisbursementService`, `documentInquiryService`, and foundational data services. Figure 4.4 shows this layering.

Frontend Services Layer

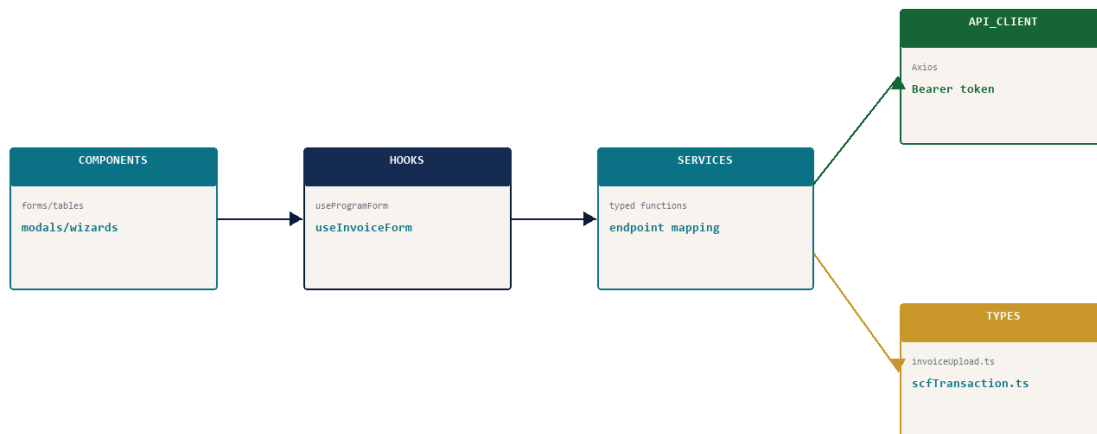


Figure 4.4: Frontend services layer

4.9.2 TypeScript Types

The frontend model is supported by TypeScript type files. `invoiceUpload.ts` defines upload files, upload rows, validation statuses, upload results, and disbursement eligibility. `scfTransaction.ts` defines transaction records, finance disbursement details, fee breakdowns, accounting entries, repayment entries, and approval history.

Table 4.1: Frontend implementation patterns

Pattern	Role in the frontend
Protected route	Ensures that sensitive pages are rendered only for authenticated users.
Business view hook	Converts Keycloak roles into BANK, ANCHOR, or COUNTERPARTY interface behavior.
Feature service	Encapsulates gateway endpoint calls behind typed frontend functions.
Wizard panes	Breaks long banking workflows into ordered, reviewable steps.
TypeScript model	Documents the expected shape of invoices, transactions, disbursements, and validation results.

4.10 Conclusion

This chapter examined the frontend implementation of the SCF platform. The React application is structured around protected routing, Keycloak integration, role-based navigation, feature modules, custom hooks, typed services, and domain-specific forms. The frontend does not merely display backend data. It shapes the user’s operational path through configuration, onboarding, invoice management, early payment, finance disbursement, transaction inquiry, and document handling.

The next chapter focuses on the Disbursement Module as the academic PFE project. It will connect the platform context, backend domain model, and frontend flows to the specific module assigned during the internship.

Chapter 5

Disbursement Module: PFE Project

5.1 Introduction

The previous chapters described the platform from the outside inward: business context, technical architecture, backend domain model, and frontend user flows. This chapter focuses on the academic project assigned during the internship: the Disbursement Module. In the SCF chain, this module is the moment where a validated financing intention becomes an operational request. The user no longer configures a product or uploads invoice data only for preparation. The user selects eligible invoices, checks the financing terms, reviews the generated accounting and repayment information, and submits the request into a governed lifecycle.

The module was not developed in isolation from the real platform. It was studied alongside daily sprint work on the Supply Chain Finance platform at Adria. At the same time, the project context imposed a clear boundary: the main platform had not yet reached the sprint milestone where the Disbursement Module would be injected natively into the architecture. The work therefore combined functional analysis, target design, UI-oriented implementation, service-contract preparation, and weekly validation with the Product Owner.

5.2 Context and Motivation

5.2.1 Position in the SCF Value Chain

Disbursement appears after several upstream conditions have been prepared. A product must exist. A program must bind the anchor, counterparties, finance limits, currencies, dates, fees, disbursement rules, and repayment rules. Invoices or cashflows must be up-

loaded, validated, and associated with the program. Only then can the financing request be assembled.

Figure 5.1 shows the target position of the module inside the SCF value chain. The Disbursement Module consumes program rules, eligible invoices, product context, and fee configuration. Its outputs are not limited to a simple payment request. They include accounting entries, a repayment schedule, and an approval lifecycle state.

Disbursement Module in the SCF Value Chain

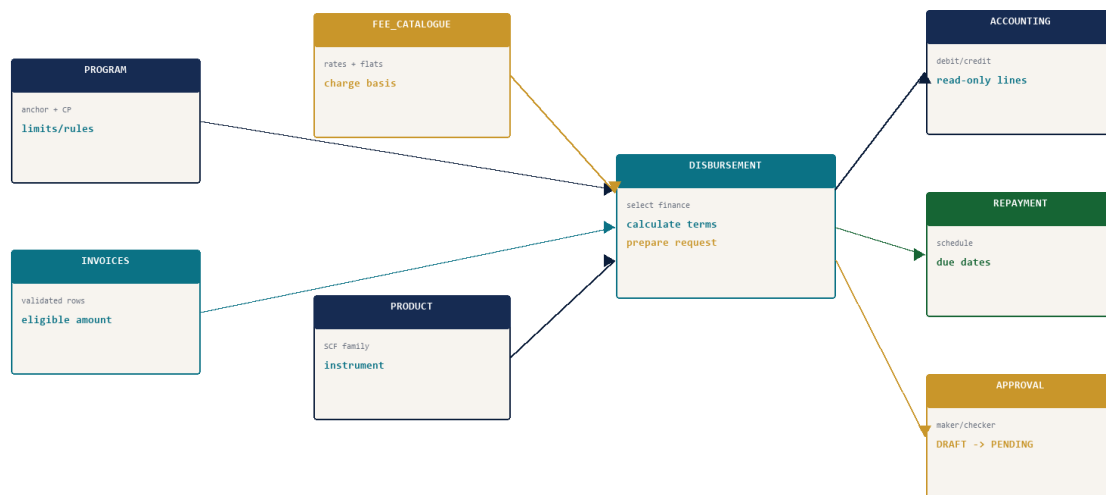


Figure 5.1: Disbursement Module position inside the SCF value chain

This position explains why the module is sensitive. A wrong program selection, an ineligible invoice, a mistaken finance percentage, or an incorrect fee basis can affect the bank's exposure and the customer's payment schedule. The interface and backend contract must therefore guide the user step by step instead of asking for an unstructured manual request.

5.2.2 Why the Module Was Treated as Parallel Work

During the internship, the SCF platform was still moving from early sprints toward a broader functional perimeter. The core team had not yet reached the integration sprint planned for the Disbursement Module. For this reason, the academic project was organized as parallel work. It advanced during available sprint gaps, after assigned Jira tickets, and through weekly refinement sessions with the Product Owner.

This setup shaped the work. The module had to remain realistic enough to match the existing platform, but flexible enough to absorb future backend decisions. The existing codebase already contained useful anchors: `FinanceDisbursementModal`, its six

panes, `invoiceDisbursementService`, `financeDisbursementService`, and the TypeScript model `FinanceDisbursement`. These elements provided a visible target for the user flow, even while the native backend integration still needed a finalized contract.

5.2.3 Relationship with the Existing Platform

The Disbursement Module depends on the same platform foundations described earlier in this report:

- authentication and role-based access through Keycloak;
- gateway routing through the `/scf/` prefix;
- program configuration and invoice/cashflow data from the SCF service;
- maker/checker governance for sensitive banking actions;
- typed frontend services built on the shared Axios client;
- transaction inquiry screens used to consult the final operational record.

Figure 5.2 summarizes the integration roadmap. The current academic report captures the design and the visible target flow. The native insertion into the production architecture remains a future sprint activity to be aligned with the final backend contract.

Disbursement Module Integration Roadmap

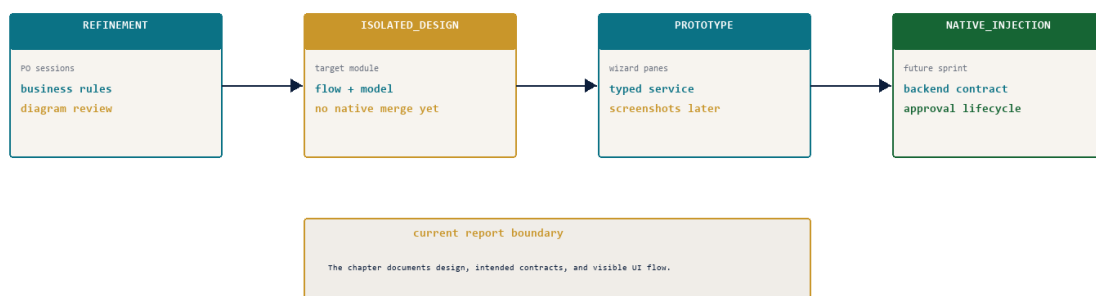


Figure 5.2: Roadmap from refinement work to native platform integration

5.3 Requirements and Functional Specification

5.3.1 Functional Requirements

The functional scope was derived from Product Owner discussions, existing platform modules, and the frontend flow already present in the repository. The target user is mainly a bank-side user, because disbursement is an operational banking action that requires control over program context, invoice eligibility, finance terms, accounting consequences, and approval status.

[Insert functional requirements validated by the Product Owner]

Table 5.1: Target functional requirements for the Disbursement Module

Requirement	Description
Program and product selection	The user selects the financing program and product context before loading invoices.
Invoice eligibility loading	The module retrieves invoices that can be considered for financing under the selected program.
Invoice selection	The user selects one or several eligible invoices to include in the request.
Finance terms capture	The module captures financing amount, dates, rates, and other terms required by the program rules.
Fee breakdown display	Fees are derived from the program fee catalogue and displayed before submission.
Accounting entry generation	Debit and credit entries are prepared and shown to the user for review.
Repayment schedule preparation	The repayment plan is configured or generated according to program rules.
Final review and submission	The user reviews the full request and submits it into a controlled lifecycle.

5.3.2 Non-functional Requirements

The module also carries non-functional expectations because it belongs to a banking platform. It must be traceable, secure, consistent, and clear under validation failure. A

user should not be able to submit a request based on missing program data or invalid invoice selection. The frontend must show the issue early, while the backend remains the final authority for validation.

Table 5.2: Non-functional expectations

Expectation	Meaning for the module
Security	Access must follow bank roles and authenticated gateway calls.
Traceability	Submitted requests must be linkable to user, program, invoices, and lifecycle actions.
Consistency	Finance terms, fees, accounting entries, and repayment schedule must be generated from the same request context.
Usability	The wizard must reduce cognitive load by separating selection, calculation, review, and submission.
Validation resilience	The UI should expose validation errors without losing the user's input when correction is possible.

5.3.3 Actors and Use Cases

The module's main actors are the bank maker or officer who prepares the disbursement, the bank checker who reviews governed actions, and the platform services that provide program, invoice, fee, and transaction data. Anchor and counterparty users may consult related transactions depending on the platform's final role matrix, but the preparation and validation of disbursement remains bank-centered in the current target.

Table 5.3: Actors involved in the Disbursement Module

Actor	Expected interaction
Bank maker or officer	Prepares the disbursement request, selects invoices, reviews computed details, and submits.
Bank checker	Reviews pending requests when maker/checker governance applies.
SCF service	Validates business rules, persists request data, and manages lifecycle transitions.
Gateway	Routes authenticated requests to the SCF service through the standard <code>/scf/</code> path.
Transaction inquiry user	Consults the resulting transaction and its related details after processing.

5.4 Design

5.4.1 Module Architecture

The target architecture follows the platform’s established pattern. The React interface uses a multi-pane wizard. The wizard calls typed service functions. The service functions use the shared Axios client, which sends authenticated requests through the gateway. The gateway routes `/scf/` calls to the SCF service. The backend then validates and persists the disbursement request using the program, invoice, fee, accounting, and repayment context.

The existing frontend already reflects this structure through `FinanceDisbursementModal`. Its panes form a natural module boundary:

- `ProgramProductSelectionPane`: selects the program and product context;
- `InvoiceSelectionPane`: loads and selects eligible invoices;
- `FinanceDetailsPane`: captures financing terms and fee breakdown;
- `AccountingEntriesPane`: displays debit and credit lines;
- `RepaymentDetailsPane`: prepares the repayment schedule;
- `ReviewSubmitPane`: reviews and submits the request.

[Insert screenshot of the Disbursement Module prototype or implemented screen]

5.4.2 Data Model

The final backend entity model for the module still needs confirmation once native integration is planned. The available TypeScript model gives a useful target shape: a finance disbursement relates to a program, a product, one or more invoice identifiers, finance terms, fee breakdowns, accounting entries, repayment entries, and approval history.

[Insert Disbursement Module data model once finalized]

Table 5.4: Target data groups for a finance disbursement

Data group	Purpose
Program and product context	Identifies the financing arrangement under which the request is made.
Selected invoices	Stores the invoices included in the financing request.
Finance terms	Captures amount, currency, value dates, rates, and calculation inputs.
Fee breakdown	Shows charges derived from the program's fee catalogue.
Accounting entries	Represents the debit and credit lines expected from the request.
Repayment schedule	Defines repayment dates, amounts, and statuses.
Approval history	Tracks maker/checker actions, comments, and status evolution.

5.4.3 API Design

The frontend service mapping identifies the target endpoint family as `/scf/finance/disbursements`. The expected creation action is represented by `financeDisbursementService.createDisbursement()`, which submits the prepared payload to the backend. The backend is expected to process the request, create a draft record, and then participate in the submission and approval lifecycle.

[Insert Disbursement API endpoints once finalized]

[Insert screenshot of Disbursement API Swagger endpoints]

Table 5.5: Target API contract candidates

Candidate endpoint	Expected purpose
POST <code>/scf/finance/disburse</code>	Create a draft disbursement request from selected invoices and finance terms.
GET <code>/scf/finance/disburse</code>	List or search finance disbursement records for inquiry and follow-up.
GET <code>/scf/finance/disburse</code>	Retrieve a single disbursement with details, fees, entries, schedule, and history.
PUT <code>/scf/finance/disburse</code>	Submit a draft request for approval.
PUT <code>/scf/finance/disburse</code>	Approve a pending request, respecting maker/checker rules.
PUT <code>/scf/finance/disburse</code>	Reject a pending request with review comments.

The table above is intentionally presented as candidate API design. The final report should replace it or confirm it after the Swagger contract is validated by the team.

5.4.4 UML Activity Diagram

Figure 5.3 presents the target activity flow. It follows a formal UML activity structure: initial node, actions, decision node with guarded branches, validation loop, final actions, and final node.

UML Activity Diagram - Disbursement Request

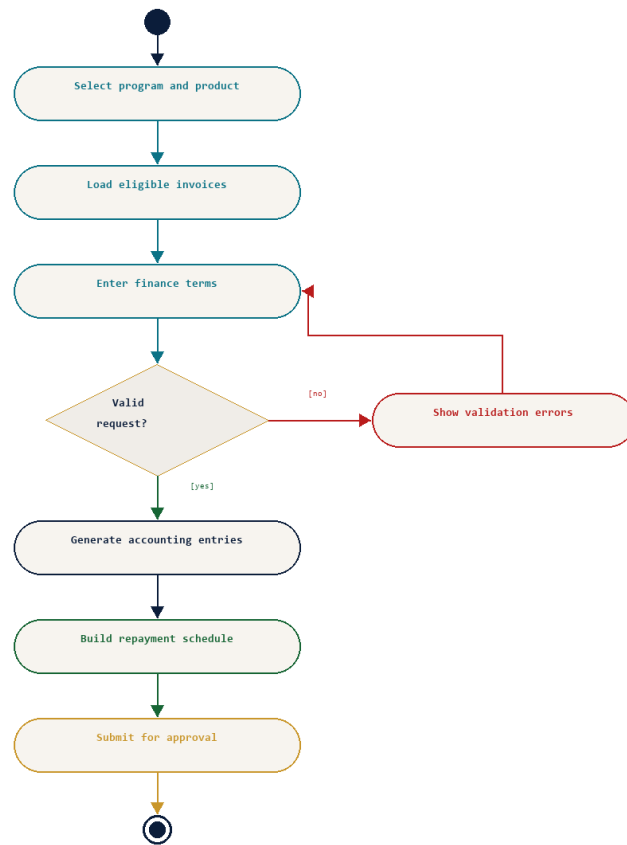


Figure 5.3: UML activity diagram for the target disbursement request flow

The activity diagram highlights the validation point. If the request is not valid, the user is sent back to correct the finance terms or invoice selection. If the request is valid, accounting entries and repayment schedule information are prepared before the request is submitted.

5.4.5 UML Sequence Diagram

Figure 5.4 shows the intended interaction between the user, frontend modal, frontend service, gateway, SCF service, and database. It is a target sequence diagram, not proof that every backend endpoint was already completed during the internship.

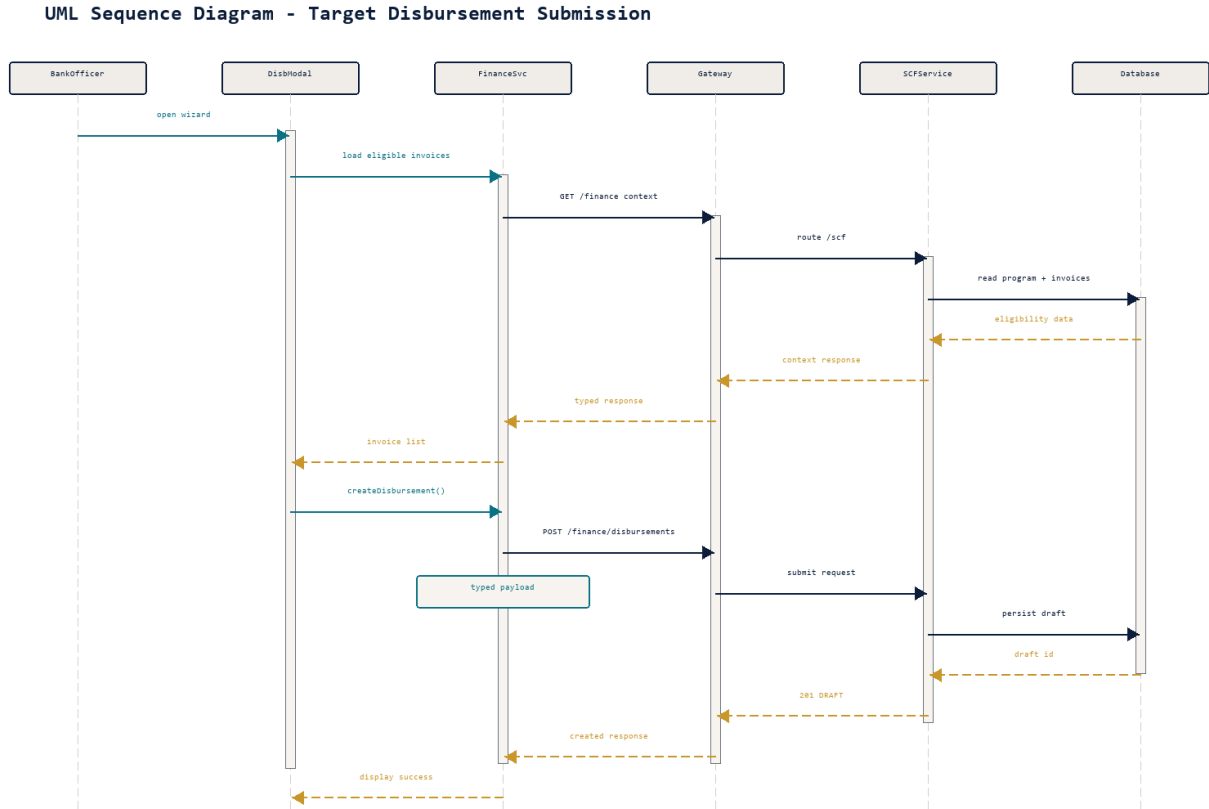


Figure 5.4: UML sequence diagram for target disbursement submission

The sequence starts with loading the context needed by the wizard. After invoice eligibility data is returned, the user builds the request in the frontend. Submission then goes through the typed service, the gateway, and the SCF service. The expected backend result is a draft request that can enter the approval lifecycle.

5.5 Implementation

5.5.1 Frontend Implementation

The frontend side is the most visible part of the module in the available project context. The wizard structure is already aligned with the business process: select context, choose invoices, review financing, inspect accounting, prepare repayment, and submit. This is a sensible design for banking operations because it prevents the user from jumping directly to submission without seeing the financial consequences of the request.

The service layer follows the same approach used by the rest of the frontend. `invoiceDisbursementService` is responsible for retrieving invoices that may be financed. `financeDisbursementService`

is responsible for creating the finance disbursement request. Both services are expected to use the shared API client, so authentication, token handling, and gateway routing remain consistent with other modules.

5.5.2 Backend Implementation

The backend implementation must remain aligned with the SCF service style described in Chapter 3. It should validate the selected program, verify invoice eligibility, compute or confirm financial terms, prepare fee information from the program fee catalogue, create accounting and repayment details, persist the request, and expose lifecycle actions under maker/checker control.

Because native integration had not yet reached the planned sprint during the internship, this section should be completed with the final classes, controllers, entities, and migrations once they are available.

[Insert backend controller, service, mapper, repository, and migration details once finalized]

5.5.3 Key Technical Decisions

Several design decisions are worth recording because they connect the academic module with the real platform:

- the module uses a wizard to match the natural order of the business operation;
- request creation is separated from approval, preserving maker/checker control;
- accounting and repayment information are displayed before submission, not hidden after it;
- the API contract follows the existing `/scf/` gateway convention;
- the module reuses program, invoice, fee, and transaction concepts rather than creating an isolated domain vocabulary.

Algorithm 2: Target creation logic for a disbursement request

Data: Selected program, product, invoices, finance terms, fee configuration

Result: Draft disbursement request ready for submission

Load program configuration and eligible invoices

Select one or more invoices

Capture finance amount, dates, and rate inputs

Validate invoice selection and finance terms against program rules

if *validation fails* **then**

 Return validation messages to the wizard

Derive fee breakdown from the program fee catalogue

Generate accounting entries and repayment schedule

Save disbursement request with draft status

5.6 Testing and Validation

Testing must cover both the user flow and the banking rules behind it. At frontend level, the wizard should be tested with complete input, missing invoice selection, invalid finance amount, and submission failure. At service level, API calls should be verified against the expected payload shape. At backend level, validation should reject requests that violate program limits, invoice eligibility, currency consistency, or maker/checker governance.

[Insert testing evidence or validation screenshots]

Table 5.6: Validation scenarios for the Disbursement Module

Scenario	Expected result
No invoice selected	The wizard prevents submission and shows a validation message.
Finance amount exceeds allowed limit	Backend validation rejects the request or returns a clear error.
Eligible invoices selected	The request can move to review with computed fees, entries, and schedule.
Draft submission	The request moves from draft preparation to the approval lifecycle.
Maker tries to approve own request	Governance rules prevent self-approval when maker/checker applies.

Validation with the Product Owner also played an important role. Functional diagrams were presented, discussed, and corrected when the flow did not match the expected business logic. This made the module less theoretical. Even before native integration, the design had to survive business review.

5.7 Conclusion

This chapter presented the Disbursement Module as the academic PFE project. The module sits at a decisive point in the SCF chain: after product, program, invoice, and fee configuration, but before accounting, repayment, transaction follow-up, and approval. The work combined target design, visible frontend flow, service-contract preparation, and Product Owner refinement while the main platform was still progressing toward native integration.

The next chapter evaluates the internship as a whole. It summarizes the contributions, the learning outcomes, the difficulties encountered, and the improvements that could support future development of the platform.

Chapter 6

Evaluation and Reflection

6.1 Introduction

This final chapter looks back at the internship from an evaluation angle. The previous chapters explained the platform, the architecture, the domain model, the frontend flows, and the Disbursement Module. Here, the focus changes slightly. The question is no longer only what the system does, but what was learned by working inside it: how the contributions were organized, what was difficult, which skills were strengthened, and what can be improved in future development.

The internship took place in a real delivery environment. Tasks were not academic exercises detached from production habits. They were assigned through Jira, implemented in a shared codebase, pushed to Bitbucket, reviewed by the technical lead, and then verified by testing. At the same time, the Disbursement Module required a more exploratory rhythm, with weekly Product Owner discussions and diagrams used as a way to refine the expected business flow.

6.2 Contributions Summary

6.2.1 Sprint-by-Sprint Overview

The internship began with two weeks of remote onboarding focused on business logic, Java, and Spring Boot. After this integration period, work continued on site inside the Supply Chain Finance Platform Team. The team was already in Sprint 2 when the internship work began and was entering Sprint 8 when the available notes were prepared. This means that the contribution period covered several active delivery cycles rather than a single isolated assignment.

[Insert screenshot or export of Jira sprint/ticket summary if available]

Table 6.1: Internship timeline and contribution focus

Period	Main focus
Weeks 1–2	Remote onboarding, business training, Java and Spring Boot refresh, understanding Adria standards.
Sprint 2 onward	Integration into the SCF team, Jira-based tasks, code review workflow, and platform familiarization.
Middle sprints	Contributions across frontend and backend modules, with regular exposure to SCF entities and user flows.
Sprint gaps and Mondays	Parallel work on the Disbursement Module through Product Owner refinement and diagram validation.
Entering Sprint 8	Consolidation of platform understanding and preparation of the academic module for future integration.

Figure 6.1 separates the two streams that shaped the internship. The first stream is delivery inside the platform team. The second is preparation of the PFE module. They were not disconnected. Daily platform work gave the Disbursement Module a realistic frame, while the academic module forced a deeper reading of SCF rules.

Internship Contribution Streams

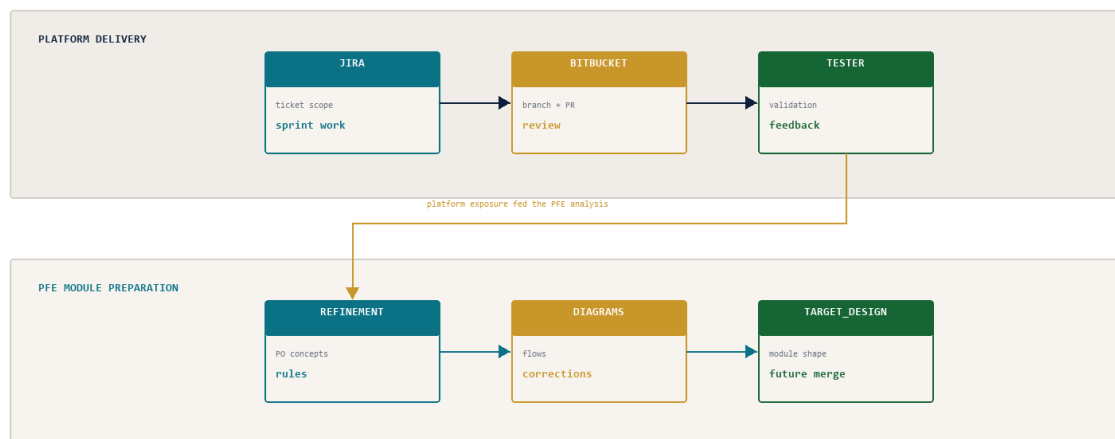


Figure 6.1: Two contribution streams during the internship

6.2.2 Classification of Contributions

The work can be grouped into four categories: user-facing features, technical tasks, bug fixes, and integration or validation tasks. The exact ticket count must be completed from Jira before final submission, but the contribution categories are already clear from the workflow.

Table 6.2: Contribution categories

Category	Examples in the internship context
User stories implemented	Work on SCF platform modules such as program configuration, counterparty onboarding, invoice handling, transaction inquiry, or finance-related screens.
Technical tasks delivered	Service integration, typed frontend services, form behavior, endpoint alignment, and code organization following platform conventions.
Bug fixes resolved	Corrections requested during review, tester feedback, validation behavior, display issues, and small inconsistencies discovered during sprint work.
Integration tasks completed	Pull request preparation, review adjustments, coordination with existing gateway, security, and service patterns.
Academic design work	Disbursement Module analysis, functional diagrams, target flow preparation, and Product Owner refinement.

[Insert quantitative summary: number of Jira tickets, pull requests, bug fixes, modules touched, and lines of code if available]

6.2.3 Quantitative Summary

Quantitative indicators should be used carefully. A line count alone says little about quality. A small correction in a validation rule may matter more than a large UI change. Still, ticket counts, pull requests, and modules touched help make the contribution visible to the reader.

Table 6.3: Quantitative indicators to complete before final submission

Indicator	To be completed
Jira tickets completed	[Insert number and ticket references]
Pull requests submitted	[Insert number and Bitbucket references]
Bug fixes validated	[Insert number or examples]
Frontend modules touched	[Insert list from repository or Jira]
Backend modules touched	[Insert list from repository or Jira]
Disbursement refinement sessions	[Insert number of Product Owner sessions if available]

6.3 Critical Evaluation

6.3.1 What Went Well

The first positive element was integration into a real Agile rhythm. Jira tickets gave concrete objectives. Pull requests created a learning loop because every contribution could receive comments from the technical lead. Tester validation added another layer of discipline: a feature was not considered finished simply because the code compiled or the interface looked correct.

The second positive element was exposure to the full platform. The internship did not stay limited to a single screen. It touched the backend model, frontend components, authentication, role-based navigation, gateway routing, document handling, invoice flows, and SCF business vocabulary. This helped connect code decisions to banking meaning.

The Disbursement Module also benefited from the team's working context. Weekly refinement sessions with the Product Owner made the academic subject less abstract. Instead of inventing a module from scratch, the work followed business explanations, diagram reviews, corrections, and platform constraints.

6.3.2 Difficulties Faced and How They Were Handled

Several difficulties appeared during the internship. Some were technical, others were organizational or domain-related. The most demanding part was not one isolated technology, but the combination of banking rules, existing architecture, security constraints, and sprint delivery pressure.

Table 6.4: Main difficulties and responses

Difficulty	Impact	Response
Banking vocabulary	SCF concepts were dense at the beginning, especially program, product, cashflow, disbursement, repayment, and maker/checker terms.	Repeated reading of existing modules, refinement discussions, and diagrams helped stabilize the vocabulary.
Existing architecture	The platform used several services, shared conventions, and pre-configured Adria standards.	Work started from existing patterns before adding or modifying code.
Maker/checker logic	Governance rules affect visibility, status transitions, and approval behavior.	The domain model and existing approval flows were studied before designing the Disbursement Module.
Role-based frontend behavior	Screens and actions change depending on bank, anchor, or counterparty roles.	Keycloak role extraction and business view helpers were used as reference points.
Parallel PFE work	The Disbursement Module had to progress before native integration was reached.	The module was treated as target design, with explicit placeholders for final backend details.

6.3.3 Limitations of the Current Work

The main limitation is the timing of the Disbursement Module. Since the platform had not yet reached the sprint planned for native integration, the module could not be presented as a fully merged backend feature with final database migrations, Swagger contract, and end-to-end test evidence. This does not remove its value, but it changes the nature of the

result. The chapter on the module must be read as a combination of validated design, visible frontend flow, and future integration preparation.

Another limitation is the lack of final quantitative evidence in the current report draft. Ticket numbers, pull request references, tester validation screenshots, and final platform screenshots must be inserted before submission. These elements will make the evaluation stronger because they connect the written report to concrete delivery artifacts.

6.4 Skills Acquired

6.4.1 Technical Skills

The internship strengthened several technical skills at once. On the backend side, the work required understanding Spring Boot, layered service structure, JPA entities, validation logic, OAuth2 resource server behavior, and service-to-service organization. On the frontend side, it developed practice with React, TypeScript, React Router, React Query, typed service layers, form decomposition, and reusable hooks.

The infrastructure dimension was also present. Docker Compose, gateway routing, Keycloak, MinIO, Gotenberg, Eureka, and Config Server were part of the platform environment. Even when not every service was modified directly, understanding their role helped make better development decisions.

Skills Acquired During the Internship

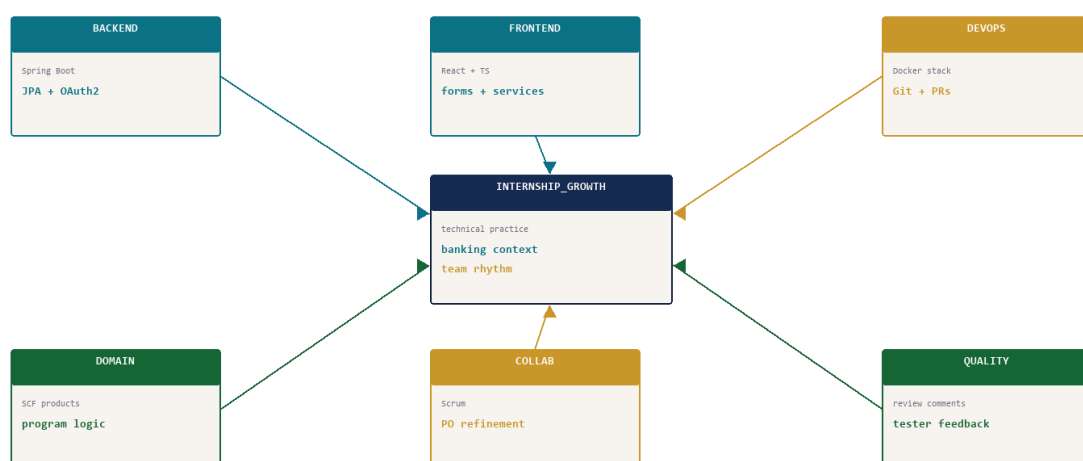


Figure 6.2: Main skills acquired during the internship

6.4.2 Professional and Soft Skills

The professional learning was just as important as the technical side. Working with Jira changed how tasks were understood: a ticket is not only an instruction, but a contract of visibility between developer, reviewer, tester, and team. Pull requests encouraged clearer code and better explanations. Review comments taught precision. Tester feedback taught patience.

Communication with the Product Owner also improved the ability to translate business speech into diagrams and system behavior. This is not automatic. A business rule may sound simple during a meeting, but it often hides conditions, exceptions, statuses, and actor responsibilities. Drawing the flow made these hidden parts visible.

Table 6.5: Skills acquired and reinforced

Skill area	Learning outcome
Backend development	Better understanding of Spring Boot services, JPA entities, validation, and governance-oriented design.
Frontend development	Stronger practice with React, TypeScript, form flows, hooks, services, and role-based UI behavior.
Security awareness	Clearer view of OAuth2, Keycloak roles, JWT validation, and protected gateway calls.
Banking domain	Practical understanding of SCF actors, programs, products, invoices, fees, disbursement, and repayment.
Collaboration	Experience with Jira, Bitbucket pull requests, technical lead review, tester validation, and Product Owner refinement.
Technical writing	Ability to transform code, diagrams, and business discussions into an academic report structure.

6.5 Recommendations for Future Development

6.5.1 Integration of the Disbursement Module

The first recommendation is to continue the native integration of the Disbursement Module into the main SCF platform. The final backend contract should confirm the entity model, controller endpoints, validation rules, approval lifecycle, and transaction inquiry linkage. Once this is done, the frontend wizard can be connected to real backend responses

instead of remaining partly target-oriented.

6.5.2 Asynchronous Processing

Some SCF operations may benefit from asynchronous processing. Invoice validation, document conversion, audit publication, notification delivery, and potentially disbursement status updates are examples where a message broker can improve decoupling. Kafka or RabbitMQ should be considered when the platform needs event-driven behavior, retry handling, or delayed processing.

6.5.3 Mobile Access for Counterparties

Counterparty users may need simpler access to invoice status, early payment requests, document notifications, and disbursement tracking. A mobile application or responsive mobile-first portal could improve the experience for SMEs and operational users who do not work all day inside a desktop banking interface.

6.5.4 Advanced Reporting and Analytics

The platform could also evolve toward richer reporting. Banks need visibility over financed amounts, overdue repayments, program utilization, counterparty exposure, product performance, and approval bottlenecks. Dashboards should not only display totals, but help users notice risk and operational delay.

Recommendations for Future Development

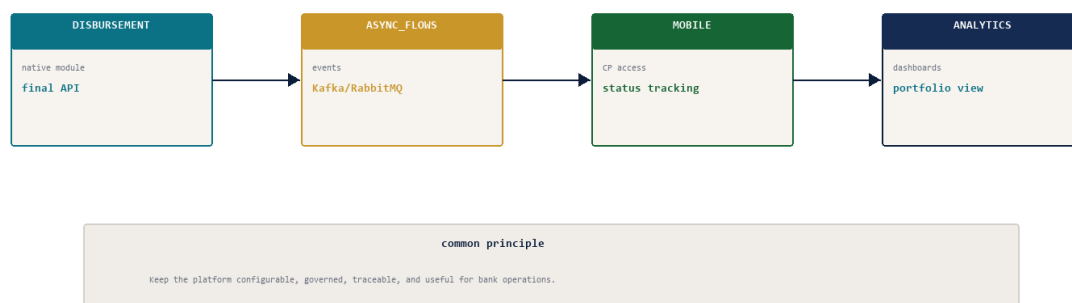


Figure 6.3: Recommendations for future platform development

6.6 Conclusion

This chapter evaluated the internship work by looking at contributions, difficulties, acquired skills, limitations, and future recommendations. The experience combined software delivery inside an Agile team with a forward-looking academic project. That combination was demanding, but it was also useful. Daily tickets gave practical discipline. The Disbursement Module gave direction. The platform context gave meaning to both.

The report can now move to the general conclusion, where the full internship journey is closed and the main outcomes are connected back to the objectives stated at the beginning.